

Physlib Monthly Report

1–31 March 2025

Contributors: Ammar Husain, Joseph Tooby-Smith, Krishna Padmasola,
Oliver Bøving, Pietro Monticone

Reviewers: Joseph Tooby-Smith, Matteo Cipollina

Generated 14 August 2026 from commit `cac586d`.

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–31 March 2025 on branch `master` of [leanprover-community/physlib](#), between commits `733bf4f` and `cac586d`. Over that period 5 contributors landed 173 commits, changing 13,090 lines across 173 files and adding 520 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	3
2	Details by file	3
2.1	<code>PhysLean.ClassicalMechanics.HarmonicOscillator.Basic</code>	3
2.2	<code>PhysLean.ClassicalMechanics.HarmonicOscillator.Solution</code>	5
2.3	<code>PhysLean.Electromagnetism.Basic</code>	8
2.4	<code>PhysLean.Electromagnetism.FieldStrength.Basic</code>	9
2.5	<code>PhysLean.Electromagnetism.FieldStrength.Derivative</code>	11
2.6	<code>PhysLean.Electromagnetism.MaxwellEquations</code>	14
2.7	<code>PhysLean.Mathematics.S03.Basic</code>	16
2.8	<code>PhysLean.Meta.Informal.Post</code>	16
2.9	<code>PhysLean.Particles.StandardModel.HiggsBoson.Basic</code>	16
2.10	<code>PhysLean.QFT.PerturbationTheory.FieldSpecification.TimeOrder</code>	16
2.11	<code>PhysLean.QuantumMechanics.FiniteTarget.Basic</code>	16
2.12	<code>PhysLean.QuantumMechanics.OneDimension.GeneralPotential.Basic</code>	17
2.13	<code>PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.TISE</code>	19
2.14	<code>PhysLean.Relativity.Lorentz.ComplexTensor.Basic</code>	19
2.15	<code>PhysLean.Relativity.Lorentz.ComplexTensor.OfRat</code>	19
2.16	<code>PhysLean.Relativity.Lorentz.Group.Basic</code>	19
2.17	<code>PhysLean.Relativity.Lorentz.Group.Boosts.Basic</code>	20
2.18	<code>PhysLean.Relativity.Lorentz.Group.Proper</code>	22
2.19	<code>PhysLean.Relativity.Lorentz.RealTensor.Basic</code>	22

2.20	<code>PhysLean.Relativity.Lorentz.RealTensor.Derivative</code>	24
2.21	<code>PhysLean.Relativity.Lorentz.RealTensor.Matrix.Pre</code>	24
2.22	<code>PhysLean.Relativity.Lorentz.RealTensor.Metrics.Basic</code>	27
2.23	<code>PhysLean.Relativity.Lorentz.RealTensor.Metrics.Pre</code>	28
2.24	<code>PhysLean.Relativity.Lorentz.RealTensor.Units.Pre</code>	29
2.25	<code>PhysLean.Relativity.Lorentz.RealTensor.Vector.Basic</code>	31
2.26	<code>PhysLean.Relativity.Lorentz.RealTensor.Vector.Boosts</code>	34
2.27	<code>PhysLean.Relativity.Lorentz.RealTensor.Vector.Causality</code>	34
2.28	<code>PhysLean.Relativity.Lorentz.RealTensor.Vector.Pre.Basic</code>	37
2.29	<code>PhysLean.Relativity.Lorentz.RealTensor.Vector.Pre.Contraction</code>	38
2.30	<code>PhysLean.Relativity.Lorentz.RealTensor.Vector.Pre.Modules</code>	38
2.31	<code>PhysLean.Relativity.SpaceTime.Basic</code>	38
2.32	<code>PhysLean.Relativity.SpaceTime.ProperTime</code>	40
2.33	<code>PhysLean.Relativity.Special.TwinParadox.Basic</code>	41
2.34	<code>PhysLean.Relativity.Tensors.OverColor.Lift</code>	42
2.35	<code>PhysLean.Relativity.Tensors.TensorSpecies.Basic</code>	42
2.36	<code>PhysLean.Relativity.Tensors.TensorSpecies.Basis</code>	43
2.37	<code>PhysLean.Relativity.Tensors.TensorSpecies.Tensor.Basic</code>	44
2.38	<code>PhysLean.Relativity.Tensors.TensorSpecies.Tensor.Contraction</code>	58
2.39	<code>PhysLean.Relativity.Tensors.TensorSpecies.Tensor.Evaluation</code>	68
2.40	<code>PhysLean.Relativity.Tensors.Tree.Basic</code>	69
2.41	<code>PhysLean.Relativity.Tensors.Tree.Elaborate</code>	69
2.42	<code>PhysLean.Relativity.Tensors.Tree.NodeIdentities.Basic</code>	72
2.43	<code>scripts.MetaPrograms.notes</code>	73
2.44	<code>scripts.MetaPrograms.TODO_to_yaml</code>	73

1 Introduction

During Mar 2025, 5 contributors submitted 173 commits that changed 13,090 lines (+9,041 / -4,049) across 173 files spanning 29 top-level areas of the repository. Of those, 28 files were newly added and 17 were removed outright. This report catalogues 520 newly added Lean declarations: 321 lemmas, 159 defs, 17 instances, 13 abbrevs, 7 structures, and 3 inductives. We're delighted to recognize the following first-time contributors this month: Ammar Husain, Krishna Padmasola, and Oliver Bøving.

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Ammar Husain	205	1
Joseph Tooby-Smith	16,563	160
Krishna Padmasola	10	1
Oliver Bøving	14	1
Pietro Monticone	759	10

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Joseph Tooby-Smith	9
Matteo Cipollina	3

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
PhysLean/Relativity	75	+15	-1	+6,166	-1,104
PhysLean/Electromagnetism	4	+4	-	+962	-0
Archive/Papers	1	-	-1	+0	-913
PhysLean/QuantumMechanics	7	+2	-	+468	-307
docs	8	-	-8	+0	-565
PhysLean/Mathematics	8	-	-	+152	-288
PhysLean/ClassicalMechanics	3	+2	-	+406	-6
PhysLean/Particles	12	-	-	+200	-130
scripts/MetaPrograms	5	-	-	+268	-40
PhysLean/QFT	18	-	-	+114	-154
docs/CuratedNotes	6	+3	-1	+34	-210
.github/workflows	1	-	-1	+0	-107
docs/_data	3	-	-	+51	-51
.github/ISSUE_TEMPLATE	2	+2	-	+81	-0
docs/_layouts	1	-	-1	+0	-52

docs/assets	3	—	-3	+0	-45
lake-manifest.json	1	—	—	+21	-21
PhysLean.lean	1	—	—	+29	-8
README.md	1	—	—	+17	-19
PhysLean/Meta	4	—	—	+16	-10
PhysLean/CondensedMatter	1	—	—	+14	-0
PhysLean/Electrodynamics	1	—	-1	+0	-12
PhysLean/Cosmology	1	—	—	+9	-0
PhysLean/Optics	1	—	—	+9	-0
PhysLean/StatisticalMechanics	1	—	—	+9	-0
PhysLean/Thermodynamics	1	—	—	+9	-0
lakefile.toml	1	—	—	+4	-5
lean-toolchain	1	—	—	+1	-1
scripts	1	—	—	+1	-1

1.4 New declarations by kind

Kind	Count
lemma	321
def	159
instance	17
abbrev	13
structure	7
inductive	3

2 Details by file

2.1 PhysLean.ClassicalMechanics.HarmonicOscillator.Basic

[PhysLean/ClassicalMechanics/HarmonicOscillator/Basic.lean](#) on GitHub.

structure HarmonicOscillator

[\[source\]](#)

The classical harmonic oscillator is specified by a mass m , and a spring constant k . Both the mass and the string constant are assumed to be positive.

```
structure HarmonicOscillator where
  m : ℝ
  k : ℝ
  m_pos : 0 < m
  k_pos : 0 < k
```

lemma k_neq_zero

[\[source\]](#)

```
@[simp]
lemma k_neq_zero : S.k ≠ 0
```

lemma m_neq_zero

[\[source\]](#)

```
@[simp]
lemma m_neq_zero : S.m ≠ 0
```

def ω [\[source\]](#)

The angular frequency of the classical harmonic oscillator, ω , is defined as $\sqrt{k/m}$.

```
noncomputable def  $\omega$  :  $\mathbb{R}$ 
```

lemma ω_{pos} [\[source\]](#)

The angular frequency of the classical harmonic oscillator is positive.

```
@[simp]
lemma  $\omega_{\text{pos}}$  :  $0 < S.\omega$ 
```

lemma ω_{sq} [\[source\]](#)

The square of the angular frequency of the classical harmonic oscillator is equal to k/m .

```
lemma  $\omega_{\text{sq}}$  :  $S.\omega^2 = S.k / S.m$ 
```

lemma $\omega_{\text{neq_zero}}$ [\[source\]](#)

The angular frequency of the classical harmonic oscillator is not equal to zero.

```
@[simp]
lemma  $\omega_{\text{neq\_zero}}$  :  $S.\omega \neq 0$ 
```

lemma $\text{inverse_}\omega_{\text{sq}}$ [\[source\]](#)

The inverse of the square of the angular frequency of the classical harmonic oscillator is m/k .

```
lemma  $\text{inverse\_}\omega_{\text{sq}}$  :  $(S.\omega^2)^{-1} = S.m/S.k$ 
```

def kineticEnergy [\[source\]](#)

The kinetic energy of the harmonic oscillator is $1/2 m (dx/dt)^2$.

```
noncomputable def kineticEnergy (x :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :  $\mathbb{R} \rightarrow \mathbb{R}$ 
```

def potentialEnergy [\[source\]](#)

The potential energy of the harmonic oscillator is $1/2 k x^2$

```
noncomputable def potentialEnergy (x :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :  $\mathbb{R} \rightarrow \mathbb{R}$ 
```

def energy [\[source\]](#)

The energy of the harmonic oscillator is the kinetic energy plus the potential energy.

```
noncomputable def energy (x :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :  $\mathbb{R} \rightarrow \mathbb{R}$ 
```

def lagrangian[\[source\]](#)

The lagrangian of the harmonic oscillator is the kinetic energy minus the potential energy.

```
noncomputable def lagrangian (x : ℝ → ℝ) : ℝ → ℝ
```

lemma lagrangian_parity[\[source\]](#)

The lagrangian of the classical harmonic oscillator obeys the condition

lagrangian S (- x) = lagrangian S x.

```
lemma lagrangian_parity (x : ℝ → ℝ) (hx : Differentiable ℝ x) :  
  lagrangian S (- x) = lagrangian S x
```

2.2 PhysLean.ClassicalMechanics.HarmonicOscillator.Solution

[PhysLean/ClassicalMechanics/HarmonicOscillator/Solution.lean](#) on GitHub.

structure InitialConditions[\[source\]](#)

The initial conditions for the harmonic oscillator specified by an initial position, and an initial velocity.

```
structure InitialConditions where
```

```
  x₀ : ℝ
```

```
  v₀ : ℝ
```

lemma ext[\[source\]](#)

```
@[ext]
```

```
lemma InitialConditions.ext {IC₁ IC₂ : InitialConditions} (h1 : IC₁.x₀ = IC₂.x₀)  
  (h2 : IC₁.v₀ = IC₂.v₀) : IC₁ = IC₂
```

def zeroIC[\[source\]](#)

The zero initial condition.

```
def zeroIC : InitialConditions
```

lemma x₀_zeroIC[\[source\]](#)

```
@[simp]
```

```
lemma x₀_zeroIC : zeroIC.x₀ = 0
```

lemma v₀_zeroIC[\[source\]](#)

```
@[simp]
```

```
lemma v₀_zeroIC : zeroIC.v₀ = 0
```

def sol[\[source\]](#)*Given initial conditions, the solution to the classical harmonic oscillator.***noncomputable def** sol (IC : InitialConditions) : $\mathbb{R} \rightarrow \mathbb{R}$ **lemma sol_eq**[\[source\]](#)**lemma** sol_eq (IC : InitialConditions) : $S.\text{sol } IC = \text{fun } t \Rightarrow IC.x_0 * \cos (S.\omega * t) + IC.v_0 / S.\omega * \sin (S.\omega * t)$ **lemma sol_zeroIC**[\[source\]](#)*For zero initial conditions, the solution is zero.***lemma** sol_zeroIC : $S.\text{sol } \text{zeroIC} = \text{fun } _ \Rightarrow 0$ **def amplitude**[\[source\]](#)*Given initial conditions, the amplitude of the classical harmonic oscillator.***noncomputable def** amplitude (IC : InitialConditions) : $\mathbb{R}$ **lemma amplitude_eq**[\[source\]](#)**lemma** amplitude_eq (IC : InitialConditions) : $S.\text{amplitude } IC = \sqrt{IC.x_0^2 + (IC.v_0 / S.\omega)^2}$ **lemma amplitude_nonneg**[\[source\]](#)*The amplitude of the classical harmonic oscillator is non-negative.*

@[simp]

lemma amplitude_nonneg (IC : InitialConditions) : $0 \leq S.\text{amplitude } IC$ **lemma amplitude_eq_norm**[\[source\]](#)**lemma** amplitude_eq_norm (IC : InitialConditions) : $S.\text{amplitude } IC = \|(\uparrow IC.x_0 + -\uparrow IC.v_0 / \uparrow S.\omega * \text{Complex.I})\|$ **lemma amplitude_sq**[\[source\]](#)**lemma** amplitude_sq (IC : InitialConditions) : $S.\text{amplitude } IC^2 = IC.x_0^2 + (IC.v_0 / S.\omega)^2$ **lemma amplitude_zeroIC**[\[source\]](#)

@[simp]

lemma amplitude_zeroIC : $S.\text{amplitude } \text{zeroIC} = 0$

lemma amplitude_eq_zero_iff_IC_eq_zeroIC[\[source\]](#)

The amplitude is zero if and only if the initial conditions are zero.

```
lemma amplitude_eq_zero_iff_IC_eq_zeroIC (IC : InitialConditions) :
  S.amplitude IC = 0 ↔ IC = zeroIC
```

def phase[\[source\]](#)

Given initial conditions, the phase of the classical harmonic oscillator.

```
noncomputable def phase (IC : InitialConditions) : ℝ
```

lemma phase_le_pi[\[source\]](#)

```
lemma phase_le_pi (IC : InitialConditions) : (S.phase IC) ≤ π
```

lemma neg_pi_lt_phase[\[source\]](#)

```
lemma neg_pi_lt_phase (IC : InitialConditions) : -π < S.phase IC
```

lemma phase_zeroIC[\[source\]](#)

```
@[simp]
lemma phase_zeroIC : S.phase zeroIC = 0
```

lemma amplitude_mul_cos_phase[\[source\]](#)

```
lemma amplitude_mul_cos_phase (IC : InitialConditions) :
  S.amplitude IC * cos (S.phase IC) = IC.x₀
```

lemma amplitude_mul_sin_phase[\[source\]](#)

```
lemma amplitude_mul_sin_phase (IC : InitialConditions) :
  S.amplitude IC * sin (S.phase IC) = - IC.v₀ / S.ω
```

lemma sol_eq_amplitude_mul_cos_phase[\[source\]](#)

```
lemma sol_eq_amplitude_mul_cos_phase (IC : InitialConditions) :
  S.sol IC = fun t => S.amplitude IC * cos (S.ω * t + S.phase IC)
```

lemma abs_sol_le_amplitude[\[source\]](#)

```
lemma abs_sol_le_amplitude (IC : InitialConditions) (t : ℝ) :
  abs (S.sol IC t) ≤ S.amplitude IC
```

lemma sol_t_zero[\[source\]](#)

```
@[simp]
lemma sol_t_zero (IC : InitialConditions) : S.sol IC 0 = IC.x₀
```

lemma sol_differentiable[\[source\]](#)

@[fun_prop]

lemma sol_differentiable (IC : InitialConditions) : Differentiable $\mathbb{R}$ (S.sol IC)**lemma sol_velocity**[\[source\]](#)

lemma sol_velocity (IC : InitialConditions) : deriv (S.sol IC) =
fun t => - IC.x₀ * S.ω * sin (S.ω * t) + IC.v₀ * cos (S.ω * t)

lemma sol_velocity_amplitude_phase[\[source\]](#)

lemma sol_velocity_amplitude_phase (IC : InitialConditions) : deriv (S.sol IC) =
fun t => - S.amplitude IC * S.ω * sin (S.ω * t + S.phase IC)

lemma sol_velocity_t_zero[\[source\]](#)

@[simp]

lemma sol_velocity_t_zero (IC : InitialConditions) : deriv (S.sol IC) 0 = IC.v₀**lemma sol_potentialEnergy**[\[source\]](#)

lemma sol_potentialEnergy (IC : InitialConditions) : S.potentialEnergy (S.sol IC) =
fun t => 1/2 * (S.k * IC.x₀ ^ 2 + S.m * IC.v₀ ^ 2) * cos (S.ω * t + S.phase IC) ^ 2

lemma sol_kineticEnergy[\[source\]](#)

lemma sol_kineticEnergy (IC : InitialConditions) : S.kineticEnergy (S.sol IC) =
fun t => 1/2 * (S.k * IC.x₀ ^ 2 + S.m * IC.v₀ ^ 2) * sin (S.ω * t + S.phase IC) ^ 2

lemma sol_energy[\[source\]](#)

lemma sol_energy (IC : InitialConditions) : S.energy (S.sol IC) =
fun _ => 1/2 * (S.m * IC.v₀ ^ 2 + S.k * IC.x₀ ^ 2)

lemma sol_lagrangian[\[source\]](#)

lemma sol_lagrangian (IC : InitialConditions) : S.lagrangian (S.sol IC) =
fun t => - 1/2 * (S.m * IC.v₀ ^ 2 + S.k * IC.x₀ ^ 2) * cos (2 * (S.ω * t + S.phase IC))

lemma sol_action[\[source\]](#)

lemma sol_action (IC : InitialConditions) (t1 t2 : $\mathbb{R}$) :
 $\int t' \text{ in } t1..t2, \text{ S.lagrangian (S.sol IC) } t' = - 1/2 * (S.m * IC.v_0^2 + S.k * IC.x_0^2) * (S.\omega^{-1} * 2^{-1} * (\sin (2 * (S.\omega * t2 + S.phase IC)) - \sin (2 * (S.\omega * t1 + S.phase IC))))$

2.3 PhysLean.Electromagnetism.Basic

PhysLean/Electromagnetism/Basic.lean on GitHub.

abbrev ElectricField[\[source\]](#)

The electric field is a map from $d+1$ dimensional spacetime to the vector space $\mathbb{R}^d$.

```
abbrev ElectricField (d : ℕ := 3)
```

abbrev MagneticField[\[source\]](#)

The magnetic field is a map from $d+1$ dimensional spacetime to the vector space $\mathbb{R}^d$.

```
abbrev MagneticField (d : ℕ := 3)
```

abbrev VectorPotential[\[source\]](#)

The vector potential of an electromagnetic field

```
abbrev VectorPotential (d : ℕ := 3)
```

2.4 PhysLean.Electromagnetism.FieldStrength.Basic

[PhysLean/Electromagnetism/FieldStrength/Basic.lean](#) on GitHub.

abbrev FieldStrength[\[source\]](#)

The Field strength is a tensor $F^\mu{}_\nu$ which is anti-symmetric..

```
noncomputable abbrev FieldStrength (d : ℕ := 3) :  
  Submodule ℝ (ℝT[d, .up] → ℝT[d, .up, .up]) where
```

lemma mem_of_repr[\[source\]](#)

```
lemma mem_of_repr {d : ℕ} {F : ℝT[d, .up, .up]}  
  (h : ∀ i j, (((realLorentzTensor d).tensorBasis _).repr F) (fun | 0 => i | 1 => j) =  
    - (((realLorentzTensor d).tensorBasis _).repr F) (fun | 0 => j | 1 => i))) :  
  {F | μ ν = - (F | ν μ)}T
```

lemma repr_symm[\[source\]](#)

```
lemma repr_symm {d : ℕ} (F : FieldStrength d) (i j : Fin (1 + d))  
  (x : ℝT[d, .up]) :  
  ((realLorentzTensor d).tensorBasis _).repr (F.1 x) (fun | 0 => i | 1 => j)  
  = - ((realLorentzTensor d).tensorBasis _).repr (F.1 x) (fun | 0 => j | 1 => i)
```

lemma repr_diag_zero[\[source\]](#)

```
@[simp]  
lemma repr_diag_zero {d : ℕ} (F : FieldStrength d) (i : Fin (1 + d))  
  (x : ℝT[d, .up]) :  
  ((realLorentzTensor d).tensorBasis _).repr (F.1 x) (fun | 0 => i | 1 => i)  
  = 0
```

def ofElectricFieldAux[\[source\]](#)

The field strength from an electric field as an element of $\mathbb{RT}[d, .up, .up]$.

```
noncomputable def ofElectricFieldAux {d : ℕ} (E : ElectricField d) (x : ℝT[d, .up]) :
  ℝT[d, .up, .up]
```

def ofElectricField[\[source\]](#)

The field strength from an electric field.

```
noncomputable def ofElectricField {d : ℕ} : ElectricField d →l[ℝ] FieldStrength d where
```

def ofMagneticFieldAux[\[source\]](#)

The field strength from a magnetic field as an element of $\mathbb{RT}[3, .up, .up]$. This is only defined here for 4d spacetime.

```
noncomputable def ofMagneticFieldAux (B : MagneticField) (x : ℝT[3, .up]) :
  ℝT[3, .up, .up]
```

def ofMagneticField[\[source\]](#)

The field strength from a magnetic field.

```
noncomputable def ofMagneticField : MagneticField →l[ℝ] FieldStrength where
```

def electricField[\[source\]](#)

The electric field given a field strength.

```
noncomputable def electricField {d : ℕ} : FieldStrength d →l[ℝ] ElectricField d where
```

lemma electricField_ofElectricField[\[source\]](#)

```
@[simp]
lemma electricField_ofElectricField {d : ℕ} (E : ElectricField d) :
  electricField (ofElectricField E) = E
```

def magneticField[\[source\]](#)

The magnetic field given a field strength.

```
noncomputable def magneticField : FieldStrength →l[ℝ] MagneticField where
```

lemma magneticField_ofMagneticField[\[source\]](#)

```
@[simp]
lemma magneticField_ofMagneticField (B : MagneticField) :
  magneticField (ofMagneticField B) = B
```

lemma electricField_ofMagneticField[\[source\]](#)

```
@[simp]
lemma electricField_ofMagneticField (B : MagneticField) :
  electricField (ofMagneticField B) = 0
```

lemma magneticField_ofElectricField[\[source\]](#)

```
@[simp]
lemma magneticField_ofElectricField (E : ElectricField) :
  magneticField (ofElectricField E) = 0
```

lemma eq_ofElectricField_add_ofMagneticField[\[source\]](#)

```
lemma eq_ofElectricField_add_ofMagneticField (F : FieldStrength) : F =
  ofElectricField (electricField F) + ofMagneticField (magneticField F)
```

def toElectricMagneticField[\[source\]](#)

The isomorphism between the field strength and the electric and magnetic fields.

```
noncomputable def toElectricMagneticField : FieldStrength  $\approx_l$   $[\mathbb{R}]$  ElectricField  $\times$  MagneticField
  where
```

abbrev fromElectricMagneticField[\[source\]](#)

The field strength from an electric and magnetic field.

```
noncomputable abbrev fromElectricMagneticField
```

lemma fromElectricMagneticField_repr[\[source\]](#)

The field strength from an electric and magnetic field written in terms of a basis.

```
lemma fromElectricMagneticField_repr (EM : ElectricField  $\times$  MagneticField) (y : SpaceTime) :
  (realLorentzTensor.tensorBasis _).repr ((fromElectricMagneticField EM).1 y) =
  Finsupp.equivFunOnFinite.symm fun b =>
    match b 0, b 1 with
    | 0, 1 => - EM.1 y 0
    | 0, 2 => - EM.1 y 1
    | 0, 3 => - EM.1 y 2
    | 1, 0 => EM.1 y 0
    | 2, 0 => EM.1 y 1
    | 3, 0 => EM.1 y 2
    | 1, 2 => - EM.2 y 2
    | 1, 3 => EM.2 y 1
    | 2, 3 => - EM.2 y 0
    | 2, 1 => EM.2 y 2
    | 3, 1 => - EM.2 y 1
    | 3, 2 => EM.2 y 0
    | _, _ => 0
```

2.5 PhysLean.Electromagnetism.FieldStrength.Derivative

[PhysLean/Electromagnetism/FieldStrength/Derivative.lean](#) on GitHub.

lemma finsupp_single_prodEquiv[\[source\]](#)

```
private lemma finsupp_single_prodEquiv (b : (j : Fin (Nat.succ 0 + (Nat.succ 0).succ)) →
  Fin (realLorentzTensor.repDim
    ((Sum.elim (fun (i : Fin 1) => realLorentzTensor.τ (![Color.up] i))
      ![Color.up, Color.up] ∘ ↗finSumFinEquiv.symm) j))) :
  (Finsupp.single (fun i => Fin.cast
    (by simp : realLorentzTensor.repDim (realLorentzTensor.τ (![Color.up] i)) =
      realLorentzTensor.repDim (![Color.up] i)) ((prodEquiv b).1 i)) (1 : ℝ)) =
    (Finsupp.single (fun | 0 => b 0) 1)
```

lemma mapTobasis_prodEquiv[\[source\]](#)

```
private lemma mapTobasis_prodEquiv (b : (j : Fin (Nat.succ 0 + (Nat.succ 0).succ)) →
  Fin (realLorentzTensor.repDim
    ((Sum.elim (fun (i : Fin 1) => realLorentzTensor.τ (![Color.up] i))
      ![Color.up, Color.up] ∘ ↗finSumFinEquiv.symm) j))) :
  (fun y => mapTobasis (↑(toElectricMagneticField.symm EM)) y
    (TensorSpecies.TensorBasis.prodEquiv b).2)
  = (fun y => mapTobasis ((toElectricMagneticField.symm EM).1) y
    (fun | (0 : Fin 2) => Fin.cast (by simp) (b 1) | (1 : Fin 2) => Fin.cast (by simp) (b 2)))
```

lemma derivative_fromElectricMagneticField_repr_diag[\[source\]](#)

```
lemma derivative_fromElectricMagneticField_repr_diag (EM : ElectricField × MagneticField)
  (hdiff : Differentiable ℝ (mapTobasis (toElectricMagneticField.symm EM).1))
  (y : SpaceTime) (j : ℕ) (hj : j < 4) :
  (realLorentzTensor.tensorBasis _).repr (∂ (fromElectricMagneticField EM).1 y)
  (fun | 0 => μ | 1 => ⟨j, hj⟩ | 2 => ⟨j, hj⟩) = 0
```

lemma derivative_fromElectricMagneticField_repr_zero_row[\[source\]](#)

```
lemma derivative_fromElectricMagneticField_repr_zero_row (EM : ElectricField × MagneticField)
  (hdiff : Differentiable ℝ (mapTobasis (toElectricMagneticField.symm EM).1))
  (y : SpaceTime) (j : ℕ) (hj : j + 1 < 4) :
  (realLorentzTensor.tensorBasis _).repr (∂ (fromElectricMagneticField EM).1 y)
  (fun | 0 => μ | 1 => ⟨0, by simp⟩ | 2 => ⟨j + 1, hj⟩) =
  - SpaceTime.deriv μ (fun y => EM.1 y ⟨j, by omega⟩) y
```

lemma derivative_fromElectricMagneticField_repr_zero_col[\[source\]](#)

```
lemma derivative_fromElectricMagneticField_repr_zero_col (EM : ElectricField × MagneticField)
  (hdiff : Differentiable ℝ (mapTobasis (toElectricMagneticField.symm EM).1))
  (y : SpaceTime) (j : ℕ) (hj : j + 1 < 4) :
  (realLorentzTensor.tensorBasis _).repr (∂ (fromElectricMagneticField EM).1 y)
  (fun | 0 => μ | 1 => ⟨j + 1, hj⟩ | 2 => ⟨0, by simp⟩) =
  SpaceTime.deriv μ (fun y => EM.1 y ⟨j, by omega⟩) y
```

lemma derivative_fromElectricMagneticField_repr_one_two[\[source\]](#)

```
lemma derivative_fromElectricMagneticField_repr_one_two (EM : ElectricField × MagneticField)
  (hdiff : Differentiable ℝ (mapTobasis (toElectricMagneticField.symm EM).1))
  (y : SpaceTime) :
  (realLorentzTensor.tensorBasis _).repr (∂ (fromElectricMagneticField EM).1 y)
  (fun | 0 => μ | 1 => ⟨1, by simp⟩ | 2 => ⟨2, by simp⟩) =
```

```
- SpaceTime.deriv μ (fun y => EM.2 y 2) y
```

lemma derivative_fromElectricMagneticField_repr_two_one

[\[source\]](#)

```
lemma derivative_fromElectricMagneticField_repr_two_one (EM : ElectricField × MagneticField)
  (hdiff :Differentiable ℝ (mapToBasis (toElectricMagneticField.symm EM).1))
  (y : SpaceTime) :
  (realLorentzTensor.tensorBasis _).repr (∂ (fromElectricMagneticField EM).1 y)
  (fun | 0 => μ | 1 => ⟨2, by simp⟩ | 2 => ⟨1, by simp⟩) =
  SpaceTime.deriv μ (fun y => EM.2 y 2) y
```

lemma derivative_fromElectricMagneticField_repr_one_three

[\[source\]](#)

```
lemma derivative_fromElectricMagneticField_repr_one_three (EM : ElectricField × MagneticField)
  (hdiff :Differentiable ℝ (mapToBasis (toElectricMagneticField.symm EM).1))
  (y : SpaceTime) :
  (realLorentzTensor.tensorBasis _).repr (∂ (fromElectricMagneticField EM).1 y)
  (fun | 0 => μ | 1 => ⟨1, by simp⟩ | 2 => ⟨3, by simp⟩) =
  SpaceTime.deriv μ (fun y => EM.2 y 1) y
```

lemma derivative_fromElectricMagneticField_repr_three_one

[\[source\]](#)

```
lemma derivative_fromElectricMagneticField_repr_three_one (EM : ElectricField × MagneticField)
  (hdiff :Differentiable ℝ (mapToBasis (toElectricMagneticField.symm EM).1))
  (y : SpaceTime) :
  (realLorentzTensor.tensorBasis _).repr (∂ (fromElectricMagneticField EM).1 y)
  (fun | 0 => μ | 1 => ⟨3, by simp⟩ | 2 => ⟨1, by simp⟩) =
  - SpaceTime.deriv μ (fun y => EM.2 y 1) y
```

lemma derivative_fromElectricMagneticField_repr_two_three

[\[source\]](#)

```
lemma derivative_fromElectricMagneticField_repr_two_three (EM : ElectricField × MagneticField)
  (hdiff :Differentiable ℝ (mapToBasis (toElectricMagneticField.symm EM).1))
  (y : SpaceTime) :
  (realLorentzTensor.tensorBasis _).repr (∂ (fromElectricMagneticField EM).1 y)
  (fun | 0 => μ | 1 => ⟨2, by simp⟩ | 2 => ⟨3, by simp⟩) =
  - SpaceTime.deriv μ (fun y => EM.2 y 3) y
```

lemma derivative_fromElectricMagneticField_repr_three_two

[\[source\]](#)

```
lemma derivative_fromElectricMagneticField_repr_three_two (EM : ElectricField × MagneticField)
  (hdiff :Differentiable ℝ (mapToBasis (toElectricMagneticField.symm EM).1))
  (y : SpaceTime) :
  (realLorentzTensor.tensorBasis _).repr (∂ (fromElectricMagneticField EM).1 y)
  (fun | 0 => μ | 1 => ⟨3, by simp⟩ | 2 => ⟨2, by simp⟩) =
  SpaceTime.deriv μ (fun y => EM.2 y 3) y
```

lemma derivative_fromElectricMagneticField_repr

[\[source\]](#)

```
lemma derivative_fromElectricMagneticField_repr (EM : ElectricField × MagneticField)
  (hdiff :Differentiable ℝ (mapToBasis (toElectricMagneticField.symm EM).1))
  (y : SpaceTime) :
  ((realLorentzTensor 3).tensorBasis _).repr (∂ (fromElectricMagneticField EM).1 y)
```

```

= Finsupp.equivFunOnFinite.symm fun b =>
  match b 0, b 1, b 2 with
  | μ, 0, 1 => - SpaceTime.deriv μ (fun y => EM.1 y 0) y
  | μ, 0, 2 => - SpaceTime.deriv μ (fun y => EM.1 y 1) y
  | μ, 0, 3 => - SpaceTime.deriv μ (fun y => EM.1 y 2) y
  | μ, 1, 0 => SpaceTime.deriv μ (fun y => EM.1 y 0) y
  | μ, 2, 0 => SpaceTime.deriv μ (fun y => EM.1 y 1) y
  | μ, 3, 0 => SpaceTime.deriv μ (fun y => EM.1 y 2) y
  | μ, 1, 2 => - SpaceTime.deriv μ (fun y => EM.2 y 2) y
  | μ, 1, 3 => SpaceTime.deriv μ (fun y => EM.2 y 1) y
  | μ, 2, 3 => - SpaceTime.deriv μ (fun y => EM.2 y 0) y
  | μ, 2, 1 => SpaceTime.deriv μ (fun y => EM.2 y 2) y
  | μ, 3, 1 => - SpaceTime.deriv μ (fun y => EM.2 y 1) y
  | μ, 3, 2 => SpaceTime.deriv μ (fun y => EM.2 y 0) y
  | _, 0, 0 => 0
  | _, 1, 1 => 0
  | _, 2, 2 => 0
  | _, 3, 3 => 0

```

lemma derivative_fromElectricMagneticField

[\[source\]](#)

```

lemma derivative_fromElectricMagneticField (EM : ElectricField × MagneticField)
  (hdiff :Differentiable ℝ (mapToBasis (toElectricMagneticField.symm EM).1)) :
  ∂ (fromElectricMagneticField EM).1 = fun (y : SpaceTime) =>
    ((reallorentzTensor 3).tensorBasis _).repr.symm <| Finsupp.equivFunOnFinite.symm fun b =>
      match b 0, b 1, b 2 with
      | μ, 0, 1 => - SpaceTime.deriv μ (fun y => EM.1 y 0) y
      | μ, 0, 2 => - SpaceTime.deriv μ (fun y => EM.1 y 1) y
      | μ, 0, 3 => - SpaceTime.deriv μ (fun y => EM.1 y 2) y
      | μ, 1, 0 => SpaceTime.deriv μ (fun y => EM.1 y 0) y
      | μ, 2, 0 => SpaceTime.deriv μ (fun y => EM.1 y 1) y
      | μ, 3, 0 => SpaceTime.deriv μ (fun y => EM.1 y 2) y
      | μ, 1, 2 => - SpaceTime.deriv μ (fun y => EM.2 y 2) y
      | μ, 1, 3 => SpaceTime.deriv μ (fun y => EM.2 y 1) y
      | μ, 2, 3 => - SpaceTime.deriv μ (fun y => EM.2 y 0) y
      | μ, 2, 1 => SpaceTime.deriv μ (fun y => EM.2 y 2) y
      | μ, 3, 1 => - SpaceTime.deriv μ (fun y => EM.2 y 1) y
      | μ, 3, 2 => SpaceTime.deriv μ (fun y => EM.2 y 0) y
      | _, 0, 0 => 0
      | _, 1, 1 => 0
      | _, 2, 2 => 0
      | _, 3, 3 => 0

```

2.6 PhysLean.Electromagnetism.MaxwellEquations

[PhysLean/Electromagnetism/MaxwellEquations.lean](#) on GitHub.

structure EMSystem

[\[source\]](#)

An electromagnetic system consists of charge density, a current density, the speed of light and the electric permittivity.

```
structure EMSystem where
```

```

  c : ℝ
  ε₀ : ℝ

```

abbrev ChargeDensity[\[source\]](#)*The charge density.***abbrev** ChargeDensity**abbrev CurrentDensity**[\[source\]](#)*Current density.***abbrev** CurrentDensity**def μ_0** [\[source\]](#)*The permeability.***noncomputable def** μ_0 : $\mathbb{R}$ **def coulombConstant**[\[source\]](#)*Coulomb's constant.***noncomputable def** coulombConstant : $\mathbb{R}$ **def GaussLawElectric**[\[source\]](#)*Gauss's law for the Electric field.***def** GaussLawElectric (E : ElectricField) : Prop**def GaussLawMagnetic**[\[source\]](#)*Gauss's law for the Magnetic field.***def** GaussLawMagnetic (B : MagneticField) : Prop**def AmpereLaw**[\[source\]](#)*Ampère's law.***def** AmpereLaw (E : ElectricField) (B : MagneticField) : Prop**def FaradayLaw**[\[source\]](#)*Faraday's law.***def** FaradayLaw (E : ElectricField) (B : MagneticField) : Prop**def MaxwellEquations**[\[source\]](#)*Maxwell's equations.***def** MaxwellEquations (E : ElectricField) (B : MagneticField) : Prop

2.7 PhysLean.Mathematics.S03.Basic

[PhysLean/Mathematics/S03/Basic.lean](#) on GitHub.

instance IsTopologicalGroup S0(3) [\[source\]](#)

The instance of a topological group on $S0(3)$, defined through the embedding of $S0(3)$ into $GL(n)$.

```
instance : IsTopologicalGroup S0(3)
```

2.8 PhysLean.Meta.Informal.Post

[PhysLean/Meta/Informal/Post.lean](#) on GitHub.

def AllInformal [\[source\]](#)

All informal results.

```
def AllInformal : CoreM (Array ConstantInfo)
```

2.9 PhysLean.Particles.StandardModel.HiggsBoson.Basic

[PhysLean/Particles/StandardModel/HiggsBoson/Basic.lean](#) on GitHub.

instance ContMDiffVectorBundle τ HiggsVec HiggsBundle (Lorentz.Vector.asSmoothMa... [\[source\]](#)

The instance of a smooth vector bundle with total space HiggsBundle and fiber HiggsVec.

```
instance : ContMDiffVectorBundle  $\tau$  HiggsVec HiggsBundle (Lorentz.Vector.asSmoothManifold 3)
```

2.10 PhysLean.QFT.PerturbationTheory.FieldSpecification.TimeOrder

[PhysLean/QFT/PerturbationTheory/FieldSpecification/TimeOrder.lean](#) on GitHub.

instance ($\phi \phi' : \mathcal{F}.FieldOp$) $\rightarrow$ Decidable (timeOrderRel $\phi \phi'$) | FieldOp.outAsymp... [\[source\]](#)

The relation timeOrderRel is decidable, but not computable so due to Real.decidableLE.

```
noncomputable instance : ( $\phi \phi' : \mathcal{F}.FieldOp$ )  $\rightarrow$  Decidable (timeOrderRel  $\phi \phi'$ )
| FieldOp.outAsymp _, _ => isTrue True.intro
| FieldOp.position  $\phi_0$ , FieldOp.position  $\phi_1$  => inferInstanceAs
  (Decidable ( $\phi_1.2$  (Sum.inl 0)  $\leq \phi_0.2$  (Sum.inl 0)))
| FieldOp.position _, FieldOp.inAsymp _ => isTrue True.intro
| FieldOp.position _, FieldOp.outAsymp _ => isFalse (fun a => a)
| FieldOp.inAsymp _, FieldOp.outAsymp _ => isFalse (fun a => a)
| FieldOp.inAsymp _, FieldOp.position _ => isFalse (fun a => a)
| FieldOp.inAsymp _, FieldOp.inAsymp _ => isTrue True.intro
```

2.11 PhysLean.QuantumMechanics.FiniteTarget.Basic

[PhysLean/QuantumMechanics/FiniteTarget/Basic.lean](#) on GitHub.

structure FiniteTarget[\[source\]](#)

A finite target quantum mechanical system with hilbert-space of dimension n and Plank constant $\hbar$ is described by a self-adjoint $n \times n$ matrix.

```
structure FiniteTarget (n : ℕ) {ħ : ℝ} (hħ : 0 < ħ) where
  H : Matrix (Fin n) (Fin n) ℂ
  H_selfAdjoint : Matrix.IsHermitian H
```

abbrev V[\[source\]](#)

The Hilbert space associated with a finite target theory A.

```
@[nolint unusedArguments]
abbrev V (_ : FiniteTarget n ħ)
```

def timeEvolutionMatrix[\[source\]](#)

*Given a finite target QM system A, the time evolution matrix for a $t : \mathbb{R}$, $A.timeEvolutionMatrix\ t$ is defined as $e^{(-I\ t/\hbar * A.H)}$.*

```
noncomputable def timeEvolutionMatrix (A : FiniteTarget n ħ) (t : ℝ) : Matrix (Fin n) (Fin n)
  ℂ
```

def timeEvolution[\[source\]](#)

Given a finite target QM system A, timeEvolution is the linear map from $A.V$ to $A.V$ obtained by multiplication with timeEvolutionMatrix.

```
noncomputable def timeEvolution (A : FiniteTarget n ħ) (t : ℝ) : A.V →l[ℂ] A.V
```

2.12 PhysLean.QuantumMechanics.OneDimension.GeneralPotential.Basic

[PhysLean/QuantumMechanics/OneDimension/GeneralPotential/Basic.lean](#) on GitHub.

def momentumOperator[\[source\]](#)

The momentum operator is defined as the map from $\mathbb{R} \rightarrow \mathbb{C}$ to $\mathbb{R} \rightarrow \mathbb{C}$ taking ψ to $-i\hbar\psi'$.

The notation $P \circ P$ can be used for the momentum operator.

```
noncomputable def momentumOperator (ħ : ℝ) (ψ : ℝ → ℂ) : ℝ → ℂ
```

lemma fun_add[\[source\]](#)

```
private lemma fun_add {α : Type*} (f g : α → ℂ) :
  (fun x => f x) + (fun x => g x) = fun x => f x + g x
```

lemma fun_smul[\[source\]](#)

```
private lemma fun_smul (a1 : ℂ) (f : ℝ → ℂ) : (a1 • fun x => f x) = (fun x => a1*(f x))
```

lemma momentumOperator_linear[\[source\]](#)

```
lemma momentumOperator_linear (ħ : ℝ) (a1 a2 : ℂ) (ψ1 ψ2 : ℝ → ℂ)
  (hψ1_x: Differentiable ℝ ψ1) (hψ2_x: Differentiable ℝ ψ2) :
  momentumOperator ħ ((a1 • ψ1) + (a2 • ψ2)) =
  a1 • momentumOperator ħ ψ1 + a2 • momentumOperator ħ ψ2
```

lemma momentumOperator_sq_linear[\[source\]](#)

```
lemma momentumOperator_sq_linear (ħ : ℝ) (a1 a2 : ℂ) (ψ1 ψ2 : ℝ → ℂ)
  (hψ1_x: Differentiable ℝ ψ1) (hψ2_x: Differentiable ℝ ψ2)
  (hψ1_xx: Differentiable ℝ (momentumOperator ħ ψ1))
  (hψ2_xx: Differentiable ℝ (momentumOperator ħ ψ2)) :
  momentumOperator ħ (momentumOperator ħ ((a1 • ψ1) + (a2 • ψ2))) =
  a1 • (momentumOperator ħ (momentumOperator ħ ψ1)) +
  a2 • (momentumOperator ħ (momentumOperator ħ ψ2))
```

def positionOperator[\[source\]](#)

The position operator is defined as the map from $\mathbb{R} \rightarrow \mathbb{C}$ to $\mathbb{R} \rightarrow \mathbb{C}$ taking ψ to $x \psi$.

```
noncomputable def positionOperator (ψ : ℝ → ℂ) : ℝ → ℂ
```

def potentialOperator[\[source\]](#)

The potential operator is defined as the map from $\mathbb{R} \rightarrow \mathbb{C}$ to $\mathbb{R} \rightarrow \mathbb{C}$ taking ψ to $V(x) \psi$.

```
noncomputable def potentialOperator (V : ℝ → ℝ) (ψ : ℝ → ℂ) : ℝ → ℂ
```

lemma potentialOperator_linear[\[source\]](#)

```
lemma potentialOperator_linear (V: ℝ → ℝ) (a1 a2 : ℂ) (ψ1 ψ2 : ℝ → ℂ) :
  potentialOperator V ((a1 • ψ1) + (a2 • ψ2)) =
  a1 • potentialOperator V ψ1 + a2 • potentialOperator V ψ2
```

structure GeneralPotential[\[source\]](#)

A quantum mechanical system in 1D is specified by a three real parameters: the mass of the particle m , a value of Planck's constant $\hbar$, and a potential function V

```
structure GeneralPotential where
```

```
  m : ℝ
  ħ : ℝ
  V : ℝ → ℝ
  ħħ : 0 < ħ
  ħm : 0 < m
```

def schrodingerOperator[\[source\]](#)

For a 1D quantum mechanical system in potential V , the Schrodinger Operator corresponding to it is defined as the function from $\mathbb{R} \rightarrow \mathbb{C}$ to $\mathbb{R} \rightarrow \mathbb{C}$ taking

*$\psi \mapsto -\hbar^2 / (2 * m) * \psi'' + V(x) * \psi$.*

```
noncomputable def schrodingerOperator (ψ : ℝ → ℂ) : ℝ → ℂ
```

lemma eval_add[\[source\]](#)

```
private lemma eval_add (f g : ℝ → ℂ) :
  (f + g) x = f x + g x
```

lemma schrodingerOperator_linear[\[source\]](#)

```
lemma schrodingerOperator_linear (a1 a2 : ℂ) (ψ1 ψ2 : ℝ → ℂ)
  (hψ1_x: Differentiable ℝ ψ1) (hψ2_x: Differentiable ℝ ψ2)
  (hψ1_xx: Differentiable ℝ (momentumOperator Q.ħ ψ1))
  (hψ2_xx: Differentiable ℝ (momentumOperator Q.ħ ψ2)) :
  schrodingerOperator Q ((a1 • ψ1) + (a2 • ψ2)) =
  a1 • schrodingerOperator Q ψ1 + a2 • schrodingerOperator Q ψ2
```

2.13 PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.TISE

[PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/TISE.lean](#) on GitHub.

lemma deriv_physHermite_characteristic_length[\[source\]](#)

```
lemma deriv_physHermite_characteristic_length (n : ℕ) :
  deriv (fun x => Complex.ofReal (physHermite n (x/Q.ξ))) = fun x =>
  Complex.ofReal (1/Q.ξ) * 2 * n * physHermite (n-1) (x/Q.ξ)
```

2.14 PhysLean.Relativity.Lorentz.ComplexTensor.Basic

[PhysLean/Relativity/Lorentz/ComplexTensor/Basic.lean](#) on GitHub.

lemma repDim_τ[\[source\]](#)

```
@[simp]
lemma repDim_τ {c : complexLorentzTensor.C} :
  complexLorentzTensor.repDim (complexLorentzTensor.τ c) = complexLorentzTensor.repDim c
```

2.15 PhysLean.Relativity.Lorentz.ComplexTensor.OfRat

[PhysLean/Relativity/Lorentz/ComplexTensor/OfRat.lean](#) on GitHub.

lemma contr_ofRat_eq_sum_contrSection[\[source\]](#)

```
lemma contr_ofRat_eq_sum_contrSection {n : ℕ} {c : Fin (n + 1 + 1) → complexLorentzTensor.C}
  {i : Fin (n + 1 + 1)}
  {j : Fin (n + 1)} {h : c (i.succAbove j) = complexLorentzTensor.τ (c i)}
  (f : (Π k, Fin (complexLorentzTensor.repDim (c k))) → RatComplexNum) :
  (contr i j h (tensorNode (ofRat f))).tensor = (tensorNode (ofRat (fun b =>
    (∑ x : TensorSpecies.TensorBasis.ContrSection b,
      f x.1 * if (x.1 i).1 = (x.1 (i.succAbove j)).1 then 1 else 0))))).tensor
```

2.16 PhysLean.Relativity.Lorentz.Group.Basic

[PhysLean/Relativity/Lorentz/Group/Basic.lean](#) on GitHub.

instance IsTopologicalGroup (LorentzGroup d)[\[source\]](#)

The embedding of the Lorentz group into $GL(n, \mathbb{R})$ gives LorentzGroup d an instance of a topological group.

```
instance : IsTopologicalGroup (LorentzGroup d)
```

2.17 PhysLean.Relativity.Lorentz.Group.Boosts.Basic

[PhysLean/Relativity/Lorentz/Group/Boosts/Basic.lean](#) on GitHub.

def γ [\[source\]](#)

The Lorentz factor (aka gamma factor or Lorentz term).

```
def  $\gamma$  ( $\beta : \mathbb{R}$ ) :  $\mathbb{R}$ 
```

lemma γ_sq [\[source\]](#)

```
lemma  $\gamma\_sq$  ( $\beta : \mathbb{R}$ ) (h $\beta : |\beta| < 1$ ) : ( $\gamma \beta$ )^2 = 1 / (1 -  $\beta^2$ )
```

lemma γ_zero [\[source\]](#)

```
@[simp]
lemma  $\gamma\_zero$  :  $\gamma 0 = 1$ 
```

lemma γ_neg [\[source\]](#)

```
@[simp]
lemma  $\gamma\_neg$  ( $\beta : \mathbb{R}$ ) :  $\gamma (-\beta) = \gamma \beta$ 
```

def boost[\[source\]](#)

The Lorentz boost with in the space direction i with speed β with $|\beta| < 1$.

```
def boost (i : Fin d) ( $\beta : \mathbb{R}$ ) (h $\beta : |\beta| < 1$ ) : LorentzGroup d
```

lemma boost_transpose_eq_self[\[source\]](#)

```
@[simp]
lemma boost_transpose_eq_self (i : Fin d) { $\beta : \mathbb{R}$ } (h $\beta : |\beta| < 1$ ) :
  transpose (boost i  $\beta$  h $\beta$ ) = boost i  $\beta$  h $\beta$ 
```

lemma boost_transpose_matrix_eq_self[\[source\]](#)

```
@[simp]
lemma boost_transpose_matrix_eq_self (i : Fin d) { $\beta : \mathbb{R}$ } (h $\beta : |\beta| < 1$ ) :
  Matrix.transpose (boost i  $\beta$  h $\beta$ ).1 = (boost i  $\beta$  h $\beta$ ).1
```

lemma boost_zero_eq_id[\[source\]](#)

@[simp]

lemma boost_zero_eq_id (i : Fin d) : boost i 0 (**by** simp) = 1**lemma boost_inverse**[\[source\]](#)**lemma** boost_inverse (i : Fin d) { $\beta : \mathbb{R}$ } (h β : $|\beta| < 1$) :
(boost i β h β)⁻¹ = boost i (- β) (**by** simp using h β)**lemma boost_inl_0_inl_0**[\[source\]](#)

@[simp]

lemma boost_inl_0_inl_0 (i : Fin d) { $\beta : \mathbb{R}$ } (h β : $|\beta| < 1$) :
(boost i β h β).1 (Sum.inl 0) (Sum.inl 0) = $\gamma \beta$ **lemma boost_inr_self_inr_self**[\[source\]](#)

@[simp]

lemma boost_inr_self_inr_self (i : Fin d) { $\beta : \mathbb{R}$ } (h β : $|\beta| < 1$) :
(boost i β h β).1 (Sum.inr i) (Sum.inr i) = $\gamma \beta$ **lemma boost_inl_0_inr_self**[\[source\]](#)

@[simp]

lemma boost_inl_0_inr_self (i : Fin d) { $\beta : \mathbb{R}$ } (h β : $|\beta| < 1$) :
(boost i β h β).1 (Sum.inl 0) (Sum.inr i) = $-\gamma \beta * \beta$ **lemma boost_inr_self_inl_0**[\[source\]](#)

@[simp]

lemma boost_inr_self_inl_0 (i : Fin d) { $\beta : \mathbb{R}$ } (h β : $|\beta| < 1$) :
(boost i β h β).1 (Sum.inr i) (Sum.inl 0) = $-\gamma \beta * \beta$ **lemma boost_inl_0_inr_other**[\[source\]](#)**lemma** boost_inl_0_inr_other {i j : Fin d} { $\beta : \mathbb{R}$ } (h β : $|\beta| < 1$) (hij : j $\neq$ i) :
(boost i β h β).1 (Sum.inl 0) (Sum.inr j) = 0**lemma boost_inr_other_inl_0**[\[source\]](#)**lemma** boost_inr_other_inl_0 {i j : Fin d} { $\beta : \mathbb{R}$ } (h β : $|\beta| < 1$) (hij : j $\neq$ i) :
(boost i β h β).1 (Sum.inr j) (Sum.inl 0) = 0**lemma boost_inr_self_inr_other**[\[source\]](#)**lemma** boost_inr_self_inr_other {i j : Fin d} { $\beta : \mathbb{R}$ } (h β : $|\beta| < 1$) (hij : j $\neq$ i) :
(boost i β h β).1 (Sum.inr i) (Sum.inr j) = 0

lemma boost_inr_other_inr_self[\[source\]](#)

```
lemma boost_inr_other_inr_self {i j : Fin d} {β : ℝ} (hβ : |β| < 1) (hij : j ≠ i) :
  (boost i β hβ).1 (Sum.inr j) (Sum.inr i) = 0
```

lemma boost_inr_other_inr[\[source\]](#)

```
lemma boost_inr_other_inr {i j k : Fin d} {β : ℝ} (hβ : |β| < 1) (hij : j ≠ i) :
  (boost i β hβ).1 (Sum.inr j) (Sum.inr k) = if j = k then 1 else 0
```

lemma boost_inr_inr_other[\[source\]](#)

```
lemma boost_inr_inr_other {i j k : Fin d} {β : ℝ} (hβ : |β| < 1) (hij : j ≠ i) :
  (boost i β hβ).1 (Sum.inr k) (Sum.inr j) = if j = k then 1 else 0
```

2.18 PhysLean.Relativity.Lorentz.Group.Proper

[PhysLean/Relativity/Lorentz/Group/Proper.lean](#) on GitHub.

instance IsTopologicalGroup ℤ₂[\[source\]](#)

The instance of a topological group on $\mathbb{Z}_2$ defined via the discrete topology.

```
instance : IsTopologicalGroup ℤ₂
```

2.19 PhysLean.Relativity.Lorentz.RealTensor.Basic

[PhysLean/Relativity/Lorentz/RealTensor/Basic.lean](#) on GitHub.

inductive Color[\[source\]](#)

The colors associated with complex representations of $SL(2, \mathbb{C})$ of interest to physics.

```
inductive Color
| up : Color
| down : Color
```

instance DecidableEq Color[\[source\]](#)

Color for complex Lorentz tensors is decidable.

```
instance : DecidableEq Color
```

def realLorentzTensor[\[source\]](#)

The tensor structure for complex Lorentz tensors.

```
def realLorentzTensor (d : ℕ := 3) : TensorSpecies ℝ where
```

instance DecidableEq (realLorentzTensor d).C[\[source\]](#)

Color for real Lorentz tensors is decidable.

```
instance (d : ℕ) : DecidableEq (realLorentzTensor d).C
```

lemma C_eq_color[\[source\]](#)

```
@[simp]
lemma C_eq_color {d : ℕ} : (realLorentzTensor d).C = Color
```

lemma repDim_up[\[source\]](#)

```
lemma repDim_up {d : ℕ} : (realLorentzTensor d).repDim Color.up = 1 + d
```

lemma repDim_down[\[source\]](#)

```
lemma repDim_down {d : ℕ} : (realLorentzTensor d).repDim Color.down = 1 + d
```

lemma repDim_eq_one_plus_dim[\[source\]](#)

```
@[simp]
lemma repDim_eq_one_plus_dim {d : ℕ} {c : (realLorentzTensor d).C} :
  (realLorentzTensor d).repDim c = 1 + d
```

lemma τ_up_eq_down[\[source\]](#)

```
@[simp]
lemma τ_up_eq_down {d : ℕ} : (realLorentzTensor d).τ Color.up = Color.down
```

lemma τ_down_eq_up[\[source\]](#)

```
@[simp]
lemma τ_down_eq_up {d : ℕ} : (realLorentzTensor d).τ Color.down = Color.up
```

lemma contr_basis[\[source\]](#)

```
lemma contr_basis {d : ℕ} {c : (realLorentzTensor d).C}
  (i : Fin ((realLorentzTensor d).repDim c))
  (j : Fin ((realLorentzTensor d).repDim ((realLorentzTensor d).τ c))) :
  (realLorentzTensor d).castToField
    (((realLorentzTensor d).contr.app (Discrete.mk c)).hom
      ((realLorentzTensor d).basis c i ⊗ₜ
        (realLorentzTensor d).basis ((realLorentzTensor d).τ c) j))
    = (if i.val = j.val then 1 else 0)
```

lemma contr_tensorBasis_repr_apply_eq_contrSection[\[source\]](#)

```
lemma contr_tensorBasis_repr_apply_eq_contrSection {n d: ℕ}
  {c : Fin (n + 1 + 1) → (realLorentzTensor d).C}
  {i : Fin (n + 1 + 1)}
  {j : Fin (n + 1)} {h : c (i.succAbove j) = (realLorentzTensor d).τ (c i)}
  (t : TensorTree (realLorentzTensor d) c)
  (b : Π k, Fin ((realLorentzTensor d).repDim (c (i.succAbove (j.succAbove k))))) :
  ((realLorentzTensor d).tensorBasis
    (c ∘ i.succAbove ∘ j.succAbove)).repr (contr i j h t).tensor b =
  ∑ (x : TensorBasis.ContrSection b),
  (((realLorentzTensor d).tensorBasis c).repr t.tensor x.1) *
  if (x.1 i).1 = (x.1 (i.succAbove j)).1 then 1 else 0
```

lemma contr_tensorBasis_repr_apply_eq_fin[\[source\]](#)

```

lemma contr_tensorBasis_repr_apply_eq_fin {n d: ℕ} {c : Fin (n + 1 + 1) → (realLorentzTensor d).C}
  {i : Fin (n + 1 + 1)}
  {j : Fin (n + 1)} {h : c (i.succAbove j) = (realLorentzTensor d).τ (c i)}
  (t : TensorTree (realLorentzTensor d) c)
  (b : Π k, Fin ((realLorentzTensor d).repDim (c (i.succAbove (j.succAbove k))))) :
  ((realLorentzTensor d).tensorBasis
   (c ∘ i.succAbove ∘ j.succAbove)).repr (contr i j h t).tensor b =
  ∑ (x : Fin (1 + d)),
  (((realLorentzTensor d).tensorBasis c).repr t.tensor
   (liftToContrSection b (Fin.cast (by simp) x, Fin.cast (by simp) x)))

```

2.20 PhysLean.Relativity.Lorentz.RealTensor.Derivative

[PhysLean/Relativity/Lorentz/RealTensor/Derivative.lean](#) on GitHub.

def mapToBasis[\[source\]](#)

The map between coordinates given a map $\mathbb{RT}(d, cm) \rightarrow \mathbb{RT}(d, cn)$.

```

noncomputable def mapToBasis {d n m : ℕ} {cm : Fin m → (realLorentzTensor d).C}
  {cn : Fin n → (realLorentzTensor d).C} (f :  $\mathbb{RT}(d, cm) \rightarrow \mathbb{RT}(d, cn)$ ) :
  (((j : Fin m) → Fin ((realLorentzTensor d).repDim (cm j))) → ℝ) →
  ((j : Fin n) → Fin ((realLorentzTensor d).repDim (cn j))) → ℝ

```

def derivative[\[source\]](#)

The derivative of a function $f : \mathbb{RT}(d, cm) \rightarrow \mathbb{RT}(d, cn)$, giving a function $\mathbb{RT}(d, cm) \rightarrow \mathbb{RT}(d, (\text{Sum.elim } cm \text{ } cn) \circ \text{finSumFinEquiv.symm})$.

```

noncomputable def derivative {d n m : ℕ} {cm : Fin m → (realLorentzTensor d).C}
  {cn : Fin n → (realLorentzTensor d).C} (f :  $\mathbb{RT}(d, cm) \rightarrow \mathbb{RT}(d, cn)$ ) :
   $\mathbb{RT}(d, cm) \rightarrow \mathbb{RT}(d, (\text{Sum.elim } (\text{fun } i \Rightarrow (\text{realLorentzTensor } d).\tau (cm \text{ } i)) \text{ } cn) \circ \text{finSumFinEquiv.symm})$ 

```

lemma derivative_repr[\[source\]](#)

```

lemma derivative_repr {d n m : ℕ} {cm : Fin m → (realLorentzTensor d).C}
  {cn : Fin n → (realLorentzTensor d).C} (f :  $\mathbb{RT}(d, cm) \rightarrow \mathbb{RT}(d, cn)$ )
  (y :  $\mathbb{RT}(d, cm)$ )
  (b : (j : Fin (m + n)) →
    Fin ((realLorentzTensor d).repDim
      (((fun i => (realLorentzTensor d).τ (cm i)) ⊗v cn) ∘ ↑finSumFinEquiv.symm) j)))
  (h1 : DifferentiableAt ℝ (mapToBasis f)
    (Finsupp.equivFunOnFinite (((realLorentzTensor d).tensorBasis cm).repr y))) :
  ((realLorentzTensor d).tensorBasis _).repr (∂ f y) b =
  fderiv ℝ (fun y => mapToBasis f y (prodEquiv b).2)
    (((realLorentzTensor d).tensorBasis cm).repr y)
    (Finsupp.single (fun i => Fin.cast (by simp) ((prodEquiv b).1 i)) (1 : ℝ))

```

2.21 PhysLean.Relativity.Lorentz.RealTensor.Matrix.Pre

[PhysLean/Relativity/Lorentz/RealTensor/Matrix/Pre.lean](#) on GitHub.

def contrContrToMatrixRe[\[source\]](#)*Equivalence of $\text{Contr} \otimes \text{Contr}$ to $(1 + d) \times (1 + d)$ real matrices.*

```
def contrContrToMatrixRe {d : ℕ} : (Contr d ⊗ Contr d).V ≈l[ℝ]
  Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ
```

lemma contrContrToMatrixRe_symm_expand_tmul[\[source\]](#)*Expanding contrContrToMatrixRe in terms of the standard basis.*

```
lemma contrContrToMatrixRe_symm_expand_tmul (M : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ) :
  contrContrToMatrixRe.symm M = ∑ i, ∑ j, M i j • (contrBasis d i ⊗t[ℝ] contrBasis d j)
```

def coCoToMatrixRe[\[source\]](#)*Equivalence of $\text{Co} \otimes \text{Co}$ to $(1 + d) \times (1 + d)$ real matrices.*

```
def coCoToMatrixRe {d : ℕ} : (Co d ⊗ Co d).V ≈l[ℝ]
  Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ
```

lemma coCoToMatrixRe_symm_expand_tmul[\[source\]](#)*Expanding coCoToMatrixRe in terms of the standard basis.*

```
lemma coCoToMatrixRe_symm_expand_tmul (M : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ) :
  coCoToMatrixRe.symm M = ∑ i, ∑ j, M i j • (coBasis d i ⊗t[ℝ] coBasis d j)
```

def contrCoToMatrixRe[\[source\]](#)*Equivalence of $\text{Contr} d \otimes \text{Co} d$ to $(1 + d) \times (1 + d)$ real matrices.*

```
def contrCoToMatrixRe {d : ℕ} : (Contr d ⊗ Co d).V ≈l[ℝ]
  Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ
```

lemma contrCoToMatrixRe_symm_expand_tmul[\[source\]](#)*Expansion of $(\text{coBasis } d)$ $(\text{coBasis } d)$ in terms of the standard basis.*

```
lemma contrCoToMatrixRe_symm_expand_tmul (M : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ) :
  contrCoToMatrixRe.symm M = ∑ i, ∑ j, M i j • (contrBasis d i ⊗t[ℝ] coBasis d j)
```

def coContrToMatrixRe[\[source\]](#)*Equivalence of $\text{Co} d \otimes \text{Contr} d$ to $(1 + d) \times (1 + d)$ real matrices.*

```
def coContrToMatrixRe : (Co d ⊗ Contr d).V ≈l[ℝ]
  Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ
```

lemma coContrToMatrixRe_symm_expand_tmul[\[source\]](#)*Expansion of coContrToMatrixRe in terms of the standard basis.*

```
lemma coContrToMatrixRe_symm_expand_tmul (M : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ) :
  coContrToMatrixRe.symm M = ∑ i, ∑ j, M i j • (coBasis d i ⊗t[ℝ] contrBasis d j)
```

lemma contrContrToMatrixRe_p[\[source\]](#)

```
lemma contrContrToMatrixRe_p {d : ℕ} (v : (Contr d ⊗ Contr d).V) (M : LorentzGroup d) :
  contrContrToMatrixRe (TensorProduct.map ((Contr d).p M) ((Contr d).p M) v) =
  M.1 * contrContrToMatrixRe v * MT
```

lemma coCoToMatrixRe_p[\[source\]](#)

```
lemma coCoToMatrixRe_p {d : ℕ} (v : ((Co d) ⊗ (Co d)).V) (M : LorentzGroup d) :
  coCoToMatrixRe (TensorProduct.map ((Co d).p M) ((Co d).p M) v) =
  M.1-1T * coCoToMatrixRe v * M-1
```

lemma contrCoToMatrixRe_p[\[source\]](#)

```
lemma contrCoToMatrixRe_p {d : ℕ} (v : ((Contr d) ⊗ (Co d)).V) (M : LorentzGroup d) :
  contrCoToMatrixRe (TensorProduct.map ((Contr d).p M) ((Co d).p M) v) =
  M.1 * contrCoToMatrixRe v * M.1-1
```

lemma coContrToMatrixRe_p[\[source\]](#)

```
lemma coContrToMatrixRe_p {d : ℕ} (v : ((Co d) ⊗ (Contr d)).V) (M : LorentzGroup d) :
  coContrToMatrixRe (TensorProduct.map ((Co d).p M) ((Contr d).p M) v) =
  M.1-1T * coContrToMatrixRe v * M.1T
```

lemma contrContrToMatrixRe_p_symm[\[source\]](#)

```
lemma contrContrToMatrixRe_p_symm {d : ℕ} (v : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ)
  (M : LorentzGroup d) :
  TensorProduct.map ((Contr d).p M) ((Contr d).p M) (contrContrToMatrixRe.symm v) =
  contrContrToMatrixRe.symm (M.1 * v * M.1T)
```

lemma coCoToMatrixRe_p_symm[\[source\]](#)

```
lemma coCoToMatrixRe_p_symm {d : ℕ} (v : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ)
  (M : LorentzGroup d) :
  TensorProduct.map ((Co d).p M) ((Co d).p M) (coCoToMatrixRe.symm v) =
  coCoToMatrixRe.symm (M.1-1T * v * M.1-1)
```

lemma contrCoToMatrixRe_p_symm[\[source\]](#)

```
lemma contrCoToMatrixRe_p_symm {d : ℕ} (v : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ)
  (M : LorentzGroup d) :
  TensorProduct.map ((Contr d).p M) ((Co d).p M) (contrCoToMatrixRe.symm v) =
  contrCoToMatrixRe.symm (M.1 * v * M.1-1)
```

lemma coContrToMatrixRe_p_symm[\[source\]](#)

```
lemma coContrToMatrixRe_p_symm {d : ℕ} (v : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ)
  (M : LorentzGroup d) :
  TensorProduct.map ((Co d).p M) ((Contr d).p M) (coContrToMatrixRe.symm v) =
  coContrToMatrixRe.symm (M.1-1T * v * M.1T)
```

2.22 PhysLean.Relativity.Lorentz.RealTensor.Metrics.Basic

[PhysLean/Relativity/Lorentz/RealTensor/Metrics/Basic.lean](#) on GitHub.

def coMetric [\[source\]](#)

The metric η_{ii} as a complex Lorentz tensor.

```
def coMetric (d : ℕ := 3)
```

def contrMetric [\[source\]](#)

The metric η^{ii} as a complex Lorentz tensor.

```
def contrMetric (d : ℕ := 3)
```

lemma tensorNode_coMetric [\[source\]](#)

The definitional tensor node relation for coMetric.

```
lemma tensorNode_coMetric {d : ℕ} : {η' d | μ v}^T.tensor =
  (TensorTree.constTwoNodeE (realLorentzTensor d)
    .down .down (Lorentz.preCoMetric d)).tensor
```

lemma tensorNode_contrMetric [\[source\]](#)

The definitional tensor node relation for contrMetric.

```
lemma tensorNode_contrMetric : {η d | μ v}^T.tensor =
  (TensorTree.constTwoNodeE (realLorentzTensor d)
    .up .up (Lorentz.preContrMetric d)).tensor
```

lemma action_coMetric [\[source\]](#)

The tensor coMetric is invariant under the action of LorentzGroup d.

```
lemma action_coMetric {d : ℕ} (g : LorentzGroup d) : {g •_a η' d | μ v}^T.tensor =
  {η' d | μ v}^T.tensor
```

lemma action_contrMetric [\[source\]](#)

The tensor contrMetric is invariant under the action of LorentzGroup d.

```
lemma action_contrMetric (g : LorentzGroup d) : {g •_a η d | μ v}^T.tensor =
  {η d | μ v}^T.tensor
```

lemma coMetric_repr_apply_eq_minkowskiMatrix [\[source\]](#)

```
lemma coMetric_repr_apply_eq_minkowskiMatrix {d : ℕ}
  (b : (j : Fin (Nat.succ 0).succ) →
    Fin ((realLorentzTensor d).repDim (![Color.down, Color.down] j))) :
  ((realLorentzTensor d).tensorBasis _).repr (coMetric d) b =
  minkowskiMatrix (finSumFinEquiv.symm (b 0)) (finSumFinEquiv.symm (b 1))
```

lemma contrMetric_repr_apply_eq_minkowskiMatrix[\[source\]](#)

```

lemma contrMetric_repr_apply_eq_minkowskiMatrix {d : ℕ}
  (b : (j : Fin (Nat.succ 0).succ) → Fin ((realLorentzTensor d).repDim (![.up, .up] j))) :
  ((realLorentzTensor d).tensorBasis _).repr (contrMetric d) b =
  minkowskiMatrix (finSumFinEquiv.symm (b 0)) (finSumFinEquiv.symm (b 1))

```

2.23 PhysLean.Relativity.Lorentz.RealTensor.Metrics.Pre

[PhysLean/Relativity/Lorentz/RealTensor/Metrics/Pre.lean](#) on GitHub.

def preContrMetricVal[\[source\]](#)

The metric η^{aa} as an element of $(\text{Contr } d \otimes \text{Contr } d).V$.

```

def preContrMetricVal (d : ℕ := 3) : (Contr d ⊗ Contr d).V

```

lemma preContrMetricVal_expand_tmul[\[source\]](#)

Expansion of preContrMetricVal into basis.

```

lemma preContrMetricVal_expand_tmul {d : ℕ} : preContrMetricVal d =
  contrBasis d (Sum.inl 0) ⊗t[ℝ] contrBasis d (Sum.inl 0) -
  ∑ i, contrBasis d (Sum.inr i) ⊗t[ℝ] contrBasis d (Sum.inr i)

```

lemma preContrMetricVal_expand_tmul_minkowskiMatrix[\[source\]](#)

```

lemma preContrMetricVal_expand_tmul_minkowskiMatrix {d : ℕ} : preContrMetricVal d =
  ∑ i, (minkowskiMatrix i i) • (contrBasis d i ⊗t[ℝ] contrBasis d i)

```

def preContrMetric[\[source\]](#)

The metric η^{aa} as a morphism $\mathbb{K}_- (\text{Rep } \mathbb{R} (\text{LorentzGroup } d)) \longrightarrow \text{Contr } d \otimes \text{Contr } d$, making its invariance under the action of LorentzGroup d .

```

def preContrMetric (d : ℕ := 3) :  $\mathbb{K}_- (\text{Rep } \mathbb{R} (\text{LorentzGroup } d)) \longrightarrow \text{Contr } d \otimes \text{Contr } d$  where

```

lemma preContrMetric_apply_one[\[source\]](#)

```

lemma preContrMetric_apply_one {d : ℕ} : (preContrMetric d).hom (1 : ℝ) = preContrMetricVal d

```

def preCoMetricVal[\[source\]](#)

The metric η_{ii} as an element of $(\text{Co } d \otimes \text{Co } d).V$.

```

def preCoMetricVal (d : ℕ := 3) : (Co d ⊗ Co d).V

```

lemma preCoMetricVal_expand_tmul[\[source\]](#)

Expansion of preCoMetricVal into basis.

```

lemma preCoMetricVal_expand_tmul {d : ℕ} : preCoMetricVal d =
  coBasis d (Sum.inl 0) ⊗t[ℝ] coBasis d (Sum.inl 0) -
  ∑ i, coBasis d (Sum.inr i) ⊗t[ℝ] coBasis d (Sum.inr i)

```

lemma preCoMetricVal_expand_tmul_minkowskiMatrix[\[source\]](#)

```
lemma preCoMetricVal_expand_tmul_minkowskiMatrix {d : ℕ} : preCoMetricVal d =
  ∑ i, (minkowskiMatrix i i) • (coBasis d i ⊗ℝ coBasis d i)
```

def preCoMetric[\[source\]](#)

The metric η_{ii} as a morphism $\mathbb{K}_- (\text{Rep } \mathbb{C} (\text{LorentzGroup } d)) \rightarrow \text{Co } d \otimes \text{Co } d$, making its invariance under the action of $\text{LorentzGroup } d$.

```
def preCoMetric (d : ℕ := 3) :  $\mathbb{K}_-$  (Rep  $\mathbb{R}$  (LorentzGroup d))  $\rightarrow$  Co d  $\otimes$  Co d where
```

lemma preCoMetric_apply_one[\[source\]](#)

```
lemma preCoMetric_apply_one {d : ℕ} : (preCoMetric d).hom (1 : ℝ) = preCoMetricVal d
```

lemma contrCoContract_apply_metric[\[source\]](#)

```
lemma contrCoContract_apply_metric {d : ℕ} : (β_ (Contr d) (Co d)).hom.hom
  (((Contr d).V  $\triangleleft$  (λ_ (Co d).V).hom)
   (((Contr d).V  $\triangleleft$  contrCoContract.hom  $\triangleright$  (Co d).V)
   (((Contr d).V  $\triangleleft$  (α_ (Contr d).V (Co d).V (Co d).V).inv)
   ((α_ (Contr d).V (Contr d).V ((Co d).V  $\otimes$  (Co d).V)).hom
   ((preContrMetric d).hom (1 : ℝ)  $\otimes_{\mathbb{R}}$  (preCoMetric d).hom (1 : ℝ)))))) =
  (preCoContrUnit d).hom (1 : ℝ)
```

lemma coContrContract_apply_metric[\[source\]](#)

```
lemma coContrContract_apply_metric {d : ℕ} : (β_ (Co d) (Contr d)).hom.hom
  (((Co d).V  $\triangleleft$  (λ_ (Contr d).V).hom)
   (((Co d).V  $\triangleleft$  coContrContract.hom  $\triangleright$  (Contr d).V)
   (((Co d).V  $\triangleleft$  (α_ (Co d).V (Contr d).V (Contr d).V).inv)
   ((α_ (Co d).V (Co d).V ((Contr d).V  $\otimes$  (Contr d).V)).hom
   ((preCoMetric d).hom (1 : ℝ)  $\otimes_{\mathbb{R}}$  (preContrMetric d).hom (1 : ℝ)))))) =
  (preContrCoUnit d).hom (1 : ℝ)
```

2.24 PhysLean.Relativity.Lorentz.RealTensor.Units.Pre

[PhysLean/Relativity/Lorentz/RealTensor/Units/Pre.lean](#) on GitHub.

def preContrCoUnitVal[\[source\]](#)

The contra-co unit for complex lorentz vectors. Usually denoted δ^i_i .

```
def preContrCoUnitVal (d : ℕ := 3) : (Contr d  $\otimes$  Co d).V
```

lemma preContrCoUnitVal_expand_tmul[\[source\]](#)

Expansion of `preContrCoUnitVal` into basis.

```
lemma preContrCoUnitVal_expand_tmul {d : ℕ} : preContrCoUnitVal d =
  ∑ i, contrBasis d i  $\otimes_{\mathbb{R}}$  coBasis d i
```

def preContrCoUnit[\[source\]](#)

The contra-co unit for complex lorentz vectors as a morphism $\mathbb{K}_-$ (Rep $\mathbb{C}$ $SL(2, \mathbb{C})$) $\longrightarrow$ $\text{complexContr} \otimes \text{complexCo}$, manifesting the invariance under the $SL(2, \mathbb{C})$ action.

```
def preContrCoUnit (d : ℕ := 3) :  $\mathbb{K}_-$  (Rep  $\mathbb{R}$  (LorentzGroup d))  $\longrightarrow$  Contr d  $\otimes$  Co d where
```

lemma preContrCoUnit_apply_one[\[source\]](#)

```
lemma preContrCoUnit_apply_one {d : ℕ} : (preContrCoUnit d).hom (1 :  $\mathbb{R}$ ) = preContrCoUnitVal d
```

def preCoContrUnitVal[\[source\]](#)

The co-contra unit for complex lorentz vectors. Usually denoted δ_i^i .

```
def preCoContrUnitVal (d : ℕ := 3) : (Co d  $\otimes$  Contr d).V
```

lemma preCoContrUnitVal_expand_tmul[\[source\]](#)

Expansion of preCoContrUnitVal into basis.

```
lemma preCoContrUnitVal_expand_tmul {d : ℕ} : preCoContrUnitVal d =  
   $\sum$  i, coBasis d i  $\otimes_{\mathbb{R}}$  contrBasis d i
```

def preCoContrUnit[\[source\]](#)

The co-contra unit for complex lorentz vectors as a morphism $\mathbb{K}_-$ (Rep $\mathbb{R}$ (LorentzGroup d)) $\longrightarrow$ Co d $\otimes$ Contr d, manifesting the invariance under the LorentzGroup d action.

```
def preCoContrUnit (d : ℕ) :  $\mathbb{K}_-$  (Rep  $\mathbb{R}$  (LorentzGroup d))  $\longrightarrow$  Co d  $\otimes$  Contr d where
```

lemma preCoContrUnit_apply_one[\[source\]](#)

```
lemma preCoContrUnit_apply_one {d : ℕ} : (preCoContrUnit d).hom (1 :  $\mathbb{R}$ ) = preCoContrUnitVal d
```

lemma contr_preContrCoUnit[\[source\]](#)

Contraction on the right with contrCoUnit does nothing.

```
lemma contr_preContrCoUnit {d : ℕ} (x : Co d) :  
  ( $\lambda$  _ (Co d)).hom.hom ((coContrContract  $\triangleright$  (Co d)).hom  
    (( $\alpha$  _ _ (Co d)).inv.hom (x  $\otimes_{\mathbb{R}}$  (preContrCoUnit d).hom (1 :  $\mathbb{R}$ )))) = x
```

lemma contr_preCoContrUnit[\[source\]](#)

Contraction on the right with coContrUnit.

```
lemma contr_preCoContrUnit {d : ℕ} (x : (Contr d)) :  
  ( $\lambda$  _ (Contr d)).hom.hom ((contrCoContract  $\triangleright$  (Contr d)).hom  
    (( $\alpha$  _ _ (Contr d)).inv.hom (x  $\otimes_{\mathbb{R}}$  (preCoContrUnit d).hom (1 :  $\mathbb{R}$ )))) = x
```

lemma preContrCoUnit_symm[\[source\]](#)

```
lemma preContrCoUnit_symm {d : ℕ} :
  ((preContrCoUnit d).hom (1 : ℝ)) = ((Contr d) <|ℝ _).hom ((β_ (Co d) (Contr d)).hom.hom
    ((preCoContrUnit d).hom (1 : ℝ)))
```

lemma preCoContrUnit_symm[\[source\]](#)

```
lemma preCoContrUnit_symm {d : ℕ} :
  ((preCoContrUnit d).hom (1 : ℝ)) = ((Co d) <|ℝ _).hom ((β_ (Contr d) (Co d)).hom.hom
    ((preContrCoUnit d).hom (1 : ℝ)))
```

2.25 PhysLean.Relativity.Lorentz.RealTensor.Vector.Basic

[PhysLean/Relativity/Lorentz/RealTensor/Vector/Basic.lean](#) on GitHub.

abbrev Vector[\[source\]](#)

Real contravariant Lorentz vector.

```
abbrev Vector (d : ℕ := 3)
```

def indexEquiv[\[source\]](#)

The equivalence between the type of indices of a Lorentz vector and $\text{Fin } 1 \oplus \text{Fin } d$.

```
def indexEquiv {d : ℕ} :
  ((j : Fin (Nat.succ 0)) → Fin ((realLorentzTensor d).repDim (![Color.up] j)))
  ≈ Fin 1 ⊕ Fin d
```

def toCoord[\[source\]](#)

The coordinates of a Lorentz vector as a linear map.

```
def toCoord {d : ℕ} : Vector d ≈ℓ[ℝ] (Fin 1 ⊕ Fin d → ℝ)
```

lemma toCoord_apply[\[source\]](#)

```
lemma toCoord_apply {d : ℕ} (p : Vector d) : toCoord p =
  (Equiv.piCngrLeft' _ indexEquiv <|
    Finsupp.equivFunOnFinite <|
    ((realLorentzTensor d).tensorBasis _).repr p)
```

lemma toCoord_injective[\[source\]](#)

```
lemma toCoord_injective {d : ℕ} : Function.Injective (@toCoord d)
```

instance CoeFun (Vector d) (fun _ => Fin 1 ⊕ Fin d → ℝ)[\[source\]](#)

```
instance : CoeFun (Vector d) (fun _ => Fin 1 ⊕ Fin d → ℝ)
```

lemma toCoord_tprod[\[source\]](#)

```

lemma toCoord_tprod {d : ℕ} (p : (i : Fin 1) →
  ↑((realLorentzTensor d).FD.obj { as := ![Color.up] i })).V) (i : Fin 1 ⊕ Fin d) :
  toCoord (PiTensorProduct.tprod ℝ p) i =
    ((Lorentz.contrBasisFin d).repr (p 0))
    (indexEquiv.symm i 0)

```

lemma tensorNode_repr_apply[\[source\]](#)

```

lemma tensorNode_repr_apply {d : ℕ} (p : Vector d)
  (b : (j : Fin (Nat.succ 0)) → Fin ((realLorentzTensor d).repDim (![Color.up] j))) :
  ((realLorentzTensor d).tensorBasis _).repr p b =
  p (finSumFinEquiv.symm (b 0))

```

def innerProduct[\[source\]](#)

The Minkowski product of Lorentz vectors in the $+ -$ convention..

```

def innerProduct {d : ℕ} (p q : Vector d) : ℝ

```

lemma innerProduct_toCoord[\[source\]](#)

```

lemma innerProduct_toCoord {d : ℕ} (p q : Vector d) :
  ⟨⟨p, q⟩⟩m = p (Sum.inl 0) * q (Sum.inl 0) - ∑ i, p (Sum.inr i) * q (Sum.inr i)

```

lemma innerProduct_zero_left[\[source\]](#)

```

@[simp]
lemma innerProduct_zero_left {d : ℕ} (q : Vector d) :
  ⟨⟨0, q⟩⟩m = 0

```

lemma innerProduct_zero_right[\[source\]](#)

```

@[simp]
lemma innerProduct_zero_right {d : ℕ} (p : Vector d) :
  ⟨⟨p, 0⟩⟩m = 0

```

lemma innerProduct_invariant[\[source\]](#)

```

@[simp]
lemma innerProduct_invariant {d : ℕ} (p q : Vector d) (Λ : LorentzGroup d) :
  ⟨⟨Λ • p, Λ • q⟩⟩m = ⟨⟨p, q⟩⟩m

```

instance FiniteDimensional ℝ (Vector d)[\[source\]](#)

```

instance : FiniteDimensional ℝ (Vector d)

```

instance isNormedAddCommGroup[\[source\]](#)

```

instance isNormedAddCommGroup (d : ℕ) : NormedAddCommGroup (Vector d)

```

instance isNormedSpace[\[source\]](#)

```
instance isNormedSpace (d : ℕ) :
  haveI
```

def toCoordContinuous[\[source\]](#)

The toCoord map as a ContinuousLinearEquiv.

```
def toCoordContinuous {d : ℕ} : Vector d ≈L[ℝ] (Fin 1 ⊕ Fin d → ℝ)
```

lemma toCoord_differentiable[\[source\]](#)

```
@[fun_prop]
lemma toCoord_differentiable {d : ℕ} :
  Differentiable ℝ (@toCoord d)
```

lemma toCoord_fderiv[\[source\]](#)

```
lemma toCoord_fderiv {d : ℕ} (x : ↑(Vector d).V) :
  (fderiv ℝ (@toCoord d) x).toLinearMap = toCoord.toLinearMap
```

def toCoordFull[\[source\]](#)

The coordinates of a Lorentz vector as a linear map.

```
def toCoordFull {d : ℕ} : Vector d ≈L[ℝ]
  (((j : Fin (Nat.succ 0)) → Fin ((realLorentzTensor d).repDim (![Color.up] j)))) → ℝ
```

lemma toCoord_apply_eq_toCoordFull_apply[\[source\]](#)

```
lemma toCoord_apply_eq_toCoordFull_apply {d : ℕ} (p : Vector d) :
  toCoord p = (Equiv.piCongrLeft' _ indexEquiv) (toCoordFull p)
```

def fromCoordFullContinuous[\[source\]](#)

The toCoordFull map as a ContinuousLinearEquiv.

```
def fromCoordFullContinuous {d : ℕ} :
  (((j : Fin (Nat.succ 0)) → Fin ((realLorentzTensor d).repDim (![Color.up] j)))) → ℝ ≈L[ℝ]
  Vector d
```

def asSmoothManifold[\[source\]](#)

The structure of a smooth manifold on Vector .

```
def asSmoothManifold (d : ℕ) : ModelWithCorners ℝ (Vector d) (Vector d)
```

instance ChartedSpace (Vector d) (Vector d)[\[source\]](#)

The instance of a ChartedSpace on Vector d.

```
instance : ChartedSpace (Vector d) (Vector d)
```

lemma action_apply_eq_sum[\[source\]](#)

```
lemma action_apply_eq_sum (i : Fin 1 ⊕ Fin d) (Λ : LorentzGroup d) (p : Vector d) :
  (Λ • p) i = ∑ j, Λ.1 i j * p j
```

lemma action_toCoord_eq_mulVec[\[source\]](#)

```
lemma action_toCoord_eq_mulVec {d} (Λ : LorentzGroup d) (p : Vector d) :
  toCoord (Λ • p) = Λ.1 *v (toCoord p)
```

2.26 PhysLean.Relativity.Lorentz.RealTensor.Vector.Boosts

[PhysLean/Relativity/Lorentz/RealTensor/Vector/Boosts.lean](#) on GitHub.

lemma boost_time_eq[\[source\]](#)

```
lemma boost_time_eq (i : Fin d) (β : ℝ) (hβ : |β| < 1) (p : Vector d) :
  (boost i β hβ • p) (Sum.inl 0) = γ β * (p (Sum.inl 0) - β * p (Sum.inr i))
```

lemma boost_inr_self_eq[\[source\]](#)

```
lemma boost_inr_self_eq (i : Fin d) (β : ℝ) (hβ : |β| < 1) (p : Vector d) :
  (boost i β hβ • p) (Sum.inr i) = γ β * (p (Sum.inr i) - β * p (Sum.inl 0))
```

lemma boost_inr_other_eq[\[source\]](#)

```
lemma boost_inr_other_eq (i j : Fin d) (hji : j ≠ i) (β : ℝ) (hβ : |β| < 1) (p : Vector d) :
  (boost i β hβ • p) (Sum.inr j) = p (Sum.inr j)
```

lemma boost_toCoord_eq[\[source\]](#)

```
lemma boost_toCoord_eq (i : Fin d) (β : ℝ) (hβ : |β| < 1) (p : Vector d) :
  toCoord (boost i β hβ • p) = fun j =>
    match j with
    | Sum.inl 0 => γ β * (p (Sum.inl 0) - β * p (Sum.inr i))
    | Sum.inr j =>
      if j = i then γ β * (p (Sum.inr i) - β * p (Sum.inl 0))
      else p (Sum.inr j)
```

2.27 PhysLean.Relativity.Lorentz.RealTensor.Vector.Causality

[PhysLean/Relativity/Lorentz/RealTensor/Vector/Causality.lean](#) on GitHub.

inductive CausalCharacter[\[source\]](#)

Classification of lorentz vectors based on their causal character.

```
inductive CausalCharacter
| timeLike
| lightLike
| spaceLike
```

def causalCharacter[\[source\]](#)

A Lorentz vector p is - lightLike if $\langle\langle p, p \rangle\rangle_m = 0$. - timeLike if $0 < \langle\langle p, p \rangle\rangle_m$. - spaceLike if $\langle\langle p, p \rangle\rangle_m < 0$. Note that $\langle\langle p, p \rangle\rangle_m$ is defined in the $+-$ convention.

```
def causalCharacter {d : ℕ} (p : Vector d) : CausalCharacter
```

lemma causalCharacter_invariant[\[source\]](#)

causalCharacter are invariant under an action of the Lorentz group.

```
lemma causalCharacter_invariant {d : ℕ} (p : Vector d)
  (Λ : LorentzGroup d) :
  causalCharacter (((realLorentzTensor d).F.obj _).p Λ p) = causalCharacter p
```

lemma timeLike_iff_norm_sq_pos[\[source\]](#)

```
lemma timeLike_iff_norm_sq_pos {d : ℕ} (p : Vector d) :
  causalCharacter p = CausalCharacter.timeLike ↔  $0 < \langle\langle p, p \rangle\rangle_m$ 
```

lemma lightLike_iff_norm_sq_zero[\[source\]](#)

```
lemma lightLike_iff_norm_sq_zero {d : ℕ} (p : Vector d) :
  causalCharacter p = CausalCharacter.lightLike ↔  $\langle\langle p, p \rangle\rangle_m = 0$ 
```

lemma spaceLike_iff_norm_sq_neg[\[source\]](#)

```
lemma spaceLike_iff_norm_sq_neg {d : ℕ} (p : Vector d) :
  causalCharacter p = CausalCharacter.spaceLike ↔  $\langle\langle p, p \rangle\rangle_m < 0$ 
```

lemma neg_causalCharacter_eq_self[\[source\]](#)

The Lorentz vector p and $-p$ have the same causalCharacter

```
lemma neg_causalCharacter_eq_self {d : ℕ} (p : Vector d) :
  causalCharacter (-p) = causalCharacter p
```

def interiorFutureLightCone[\[source\]](#)

The future light cone of a Lorentz vector p is defined as those vectors q such that - causalCharacter $(q - p)$ is timeLike and $-(q - p)$ (Sum.inl 0) is positive.

```
def interiorFutureLightCone {d : ℕ} (p : Vector d) : Set (Vector d)
```

def interiorPastLightCone[\[source\]](#)

The backward light cone of a Lorentz vector p is defined as those vectors q such that - causalCharacter $(q - p)$ is timeLike and $-(q - p)$ (Sum.inl 0) is negative.

```
def interiorPastLightCone {d : ℕ} (p : Vector d) : Set (Vector d)
```

def lightConeBoundary[\[source\]](#)

The light cone boundary (null surface) of a spacetime point p .

```
def lightConeBoundary {d : ℕ} (p : Vector d) : Set (Vector d)
```

def futureLightConeBoundary[\[source\]](#)

The future light cone boundary (null surface) of a spacetime point p .

```
def futureLightConeBoundary {d : ℕ} (p : Vector d) : Set (Vector d)
```

def pastLightConeBoundary[\[source\]](#)

The past light cone boundary (null surface) of a spacetime point p .

```
def pastLightConeBoundary {d : ℕ} (p : Vector d) : Set (Vector d)
```

lemma self_mem_lightConeBoundary[\[source\]](#)

```
lemma self_mem_lightConeBoundary {d : ℕ} (p : Vector d) : p ∈ lightConeBoundary p
```

def causallyFollows[\[source\]](#)

A proposition which is true if q is in the causal future of event p .

```
def causallyFollows {d : ℕ} (p q : Vector d) : Prop
```

def causallyPrecedes[\[source\]](#)

A proposition which is true if q is in the causal past of event p .

```
def causallyPrecedes {d : ℕ} (p q : Vector d) : Prop
```

def causallyRelated[\[source\]](#)

Events p and q are causally related.

```
def causallyRelated {d : ℕ} (p q : Vector d) : Prop
```

def causallyUnrelated[\[source\]](#)

Events p and q are causally unrelated (spacelike separated).

```
def causallyUnrelated {d : ℕ} (p q : Vector d) : Prop
```

def causalDiamond[\[source\]](#)

The causal diamond between events p and q , where p is assumed to causally precede q .

```
def causalDiamond {d : ℕ} (p q : Vector d) : Set (Vector d)
```

2.28 PhysLean.Relativity.Lorentz.RealTensor.Vector.Pre.Basic

[PhysLean/Relativity/Lorentz/RealTensor/Vector/Pre/Basic.lean](#) on GitHub.

def contrBasis

[\[source\]](#)

The standard basis of contravariant Lorentz vectors.

```
def contrBasis (d : ℕ := 3) : Basis (Fin 1 ⊕ Fin d) ℝ (Contr d)
```

lemma contrBasis_p_apply

[\[source\]](#)

@[simp]

```
lemma contrBasis_p_apply {d : ℕ} (M : LorentzGroup d) (i j : Fin 1 ⊕ Fin d) :
  (LinearMap.toMatrix (contrBasis d) (contrBasis d)) ((Contr d).p M) i j =
    M.1 i j
```

lemma contrBasis_toFinldℝ

[\[source\]](#)

@[simp]

```
lemma contrBasis_toFinldℝ {d : ℕ} (i : Fin (1) ⊕ Fin d) :
  (contrBasis d i).toFinldℝ = Pi.single i 1
```

def contrBasisFin

[\[source\]](#)

The standard basis of contravariant Lorentz vectors indexed by Fin (1 + d).

```
def contrBasisFin (d : ℕ := 3) : Basis (Fin (1 + d)) ℝ (Contr d)
```

lemma contrBasisFin_toFinldℝ

[\[source\]](#)

@[simp]

```
lemma contrBasisFin_toFinldℝ {d : ℕ} (i : Fin (1 + d)) :
  (contrBasisFin d i).toFinldℝ = Pi.single (finSumFinEquiv.symm i) 1
```

lemma contrBasisFin_repr_apply

[\[source\]](#)

```
lemma contrBasisFin_repr_apply {d : ℕ} (p : Contr d) (i : Fin (1 + d)) :
  (contrBasisFin d).repr p i = p.val (finSumFinEquiv.symm i)
```

def coBasis

[\[source\]](#)

The standard basis of contravariant Lorentz vectors.

```
def coBasis (d : ℕ := 3) : Basis (Fin 1 ⊕ Fin d) ℝ (Co d)
```

lemma coBasis_p_apply

[\[source\]](#)

@[simp]

```
lemma coBasis_p_apply {d : ℕ} (M : LorentzGroup d) (i j : Fin 1 ⊕ Fin d) :
  (LinearMap.toMatrix (coBasis d) (coBasis d)) ((Co d).p M) i j =
    M.-1T i j
```

lemma coBasis_toFinldℝ[\[source\]](#)

```
@[simp]
lemma coBasis_toFinldℝ {d : ℕ} (i : Fin (1) ⊗ Fin d) :
  (coBasis d i).toFinldℝ = Pi.single i 1
```

def coBasisFin[\[source\]](#)

The standard basis of covaraitn Lorentz vectors indexed by $\text{Fin } (1 + d)$.

```
def coBasisFin (d : ℕ := 3) : Basis (Fin (1 + d)) ℝ (Co d)
```

lemma coBasisFin_toFinldℝ[\[source\]](#)

```
@[simp]
lemma coBasisFin_toFinldℝ {d : ℕ} (i : Fin (1 + d)) :
  (coBasisFin d i).toFinldℝ = Pi.single (finSumFinEquiv.symm i) 1
```

2.29 PhysLean.Relativity.Lorentz.RealTensor.Vector.Pre.Contraction

[PhysLean/Relativity/Lorentz/RealTensor/Vector/Pre/Contraction.lean](#) on GitHub.

lemma contrCoContract_basis[\[source\]](#)

```
lemma contrCoContract_basis {d : ℕ} (i j : Fin 1 ⊗ Fin d) :
  contrCoContract.hom (contrBasis d i ⊗ₜ coBasis d j) = if i = j then (1 : ℝ) else 0
```

lemma coContrContract_basis[\[source\]](#)

```
lemma coContrContract_basis {d : ℕ} (i j : Fin 1 ⊗ Fin d) :
  coContrContract.hom (coBasis d i ⊗ₜ[ℝ] contrBasis d j) = if i = j then (1 : ℝ) else 0
```

2.30 PhysLean.Relativity.Lorentz.RealTensor.Vector.Pre.Modules

[PhysLean/Relativity/Lorentz/RealTensor/Vector/Pre/Modules.lean](#) on GitHub.

lemma mulVec_val[\[source\]](#)

```
@[simp]
lemma mulVec_val (M : Matrix (Fin 1 ⊗ Fin d) (Fin 1 ⊗ Fin d) ℝ) (v : ContrMod d) :
  (M *ₐ v).val = M *ₐ v.val
```

2.31 PhysLean.Relativity.SpaceTime.Basic

[PhysLean/Relativity/SpaceTime/Basic.lean](#) on GitHub.

abbrev Space[\[source\]](#)

The type $\text{Space } d$ representes d dimensional Euclidean space. The default value of d is 3. Thus $\text{Space} = \text{Space } 3$.

```
abbrev Space (d : ℕ := 3)
```

def coord[\[source\]](#)

For a given $\mu : \text{Fin } (1 + d)$ $\text{coord } \mu p$ is the coordinate of p in the direction μ .

This is denoted $\S \mu p$, where $\S$ is typed with $\backslash\{\}MCx$.

```
def coord {d : ℕ} (μ : Fin (1 + d)) : SpaceTime d →l[ℝ] ℝ where
```

lemma coord_apply[\[source\]](#)

```
lemma coord_apply {d : ℕ} (μ : Fin (1 + d)) (y : SpaceTime d) :
  § μ y = y (finSumFinEquiv.symm μ)
```

lemma coord_on_repr[\[source\]](#)

```
lemma coord_on_repr {d : ℕ} (μ : Fin (1 + d))
  (y : ((j : Fin (Nat.succ 0)) → Fin ((realLorentzTensor d).repDim (![Color.up] j))) → ℝ) :
  § μ (((realLorentzTensor d).tensorBasis ![Color.up]).repr.symm
    (Finsupp.equivFunOnFinite.symm y)) =
  y (fun _ => Fin.cast (by simp) μ)
```

def deriv[\[source\]](#)

The derivative of a function $\text{SpaceTime } d \rightarrow \mathbb{R}$ along the μ coordinte.

```
noncomputable def deriv {M : Type} [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
  {d : ℕ} (μ : Fin (1 + d)) (f : SpaceTime d → M) : SpaceTime d → M
```

lemma deriv_eq[\[source\]](#)

```
lemma deriv_eq {d : ℕ} (μ : Fin (1 + d)) (f : SpaceTime d → ℝ) (y : SpaceTime d) :
  SpaceTime.deriv μ f y =
  fderiv ℝ f y ((realLorentzTensor d).tensorBasis _ (fun x => Fin.cast (by simp) μ))
```

lemma deriv_zero[\[source\]](#)

```
@[simp]
lemma deriv_zero {d : ℕ} (μ : Fin (1 + d)) : SpaceTime.deriv μ (fun _ => (0 : ℝ)) = 0
```

lemma deriv_eq_deriv_on_coord[\[source\]](#)

```
lemma deriv_eq_deriv_on_coord {d : ℕ} (μ : Fin (1 + d)) (f : SpaceTime d → ℝ) (y : SpaceTime d)
  :
  SpaceTime.deriv μ f y = fderiv ℝ
    (fun y => (f (((realLorentzTensor d).tensorBasis ![Color.up]).repr.symm
      (Finsupp.equivFunOnFinite.symm y))))
    ↑(((realLorentzTensor d).tensorBasis ![Color.up]).repr y)
    ↑(Finsupp.single (fun x => Fin.cast (by simp) μ) 1)
```

lemma neg_deriv[\[source\]](#)

```
lemma neg_deriv {d : ℕ} (μ : Fin (1 + d)) (f : SpaceTime d → ℝ) :
  - SpaceTime.deriv μ f = SpaceTime.deriv μ (fun y => - f y)
```

lemma neg_deriv_apply[\[source\]](#)

```
lemma neg_deriv_apply {d : ℕ} (μ : Fin (1 + d)) (f : SpaceTime d → ℝ) (y : SpaceTime d) :
  - SpaceTime.deriv μ f y = SpaceTime.deriv μ (fun y => - f y) y
```

lemma coord_differentiable[\[source\]](#)

@[fun_prop]

```
lemma coord_differentiable {d : ℕ} (μ : Fin (1 + d)) :
  Differentiable ℝ (§ μ)
```

lemma deriv_coord_same[\[source\]](#)

```
lemma deriv_coord_same {d : ℕ} (μ : Fin (1 + d)) (y : SpaceTime d) :
  SpaceTime.deriv μ (§ μ) y = 1
```

lemma deriv_coord_diff[\[source\]](#)

```
lemma deriv_coord_diff {d : ℕ} (μ v : Fin (1 + d)) (h : μ ≠ v) (y : SpaceTime d) :
  SpaceTime.deriv μ (§ v) y = 0
```

lemma deriv_coord_eq_if[\[source\]](#)

```
lemma deriv_coord_eq_if {d : ℕ} (μ v : Fin (1 + d)) (y : SpaceTime d) :
  SpaceTime.deriv μ (§ v) y = if μ = v then 1 else 0
```

def spaceDiv[\[source\]](#)

The divergence of a function $\text{SpaceTime } d \rightarrow \text{EuclideanSpace } \mathbb{R} \text{ (Fin } d)$.

```
noncomputable def spaceDiv {d : ℕ} (f : SpaceTime d → EuclideanSpace ℝ (Fin d)) :
  SpaceTime d → ℝ
```

def spaceCurl[\[source\]](#)

The curl of a function $\text{SpaceTime} \rightarrow \text{EuclideanSpace } \mathbb{R} \text{ (Fin 3)}$.

```
def spaceCurl (f : SpaceTime → EuclideanSpace ℝ (Fin 3)) :
  SpaceTime → EuclideanSpace ℝ (Fin 3)
```

2.32 PhysLean.Relativity.SpaceTime.ProperTime

[PhysLean/Relativity/SpaceTime/ProperTime.lean](#) on GitHub.

def properTime[\[source\]](#)

The proper time from q to p . Defaults to zero if p and q have a space-like speration.

```
def properTime {d : ℕ} (q p : SpaceTime d) : ℝ
```

lemma properTime_pos_ofTimeLike[\[source\]](#)

```
lemma properTime_pos_ofTimeLike {d : ℕ} (q p : SpaceTime d)
  (h : causalCharacter (p - q) = .timeLike) :
  0 < properTime q p
```

lemma properTime_zero_ofLightLike[\[source\]](#)

```
lemma properTime_zero_ofLightLike {d : ℕ} (q p : SpaceTime d)
  (h : causalCharacter (p - q) = .lightLike) :
  properTime q p = 0
```

lemma properTime_zero_ofSpaceLike[\[source\]](#)

```
lemma properTime_zero_ofSpaceLike {d : ℕ} (q p : SpaceTime d)
  (h : causalCharacter (p - q) = .spaceLike) :
  properTime q p = 0
```

2.33 PhysLean.Relativity.Special.TwinParadox.Basic

[PhysLean/Relativity/Special/TwinParadox/Basic.lean](#) on GitHub.

structure InstantaneousTwinParadox[\[source\]](#)

The twin paradox assuming instantaneous acceleration.

```
structure InstantaneousTwinParadox where
  startPoint : SpaceTime 3
  endPoint : SpaceTime 3
  twinBMid : SpaceTime 3
  endPoint_causallyFollows_startPoint : causallyFollows startPoint endPoint
  twinBMid_causallyFollows_startPoint : causallyFollows startPoint twinBMid
  endPoint_causallyFollows_twinBMid : causallyFollows twinBMid endPoint
```

def properTimeTwinA[\[source\]](#)

The proper time experienced by twin A travelling at constant speed from T.startPoint to T.endPoint.

```
def properTimeTwinA : ℝ
```

def properTimeTwinB[\[source\]](#)

The proper time experienced by twin B travelling at constant speed from T.startPoint to T.twinBMid, and then from T.twinBMid to T.endPoint.

```
def properTimeTwinB : ℝ
```

def ageGap[\[source\]](#)

The proper time of twin A minus the proper time of twin B.

```
def ageGap : ℝ
```

def example1 [\[source\]](#)

The twin paradox in which: - Twin A starts at 0 and travels at constant speed to [15, 0, 0, 0]. - Twin B starts at 0 and travels at constant speed to [7.5, 6, 0, 0] and then at (different) constant speed to [15, 0, 0, 0].

def example1 : InstantaneousTwinParadox **where**

lemma example1_properTimeTwinA [\[source\]](#)

@[simp]

lemma example1_properTimeTwinA : example1.properTimeTwinA = 15

lemma example1_properTimeTwinB [\[source\]](#)

@[simp]

lemma example1_properTimeTwinB : example1.properTimeTwinB = 9

lemma example1_ageGap [\[source\]](#)

lemma example1_ageGap : example1.ageGap = 6

2.34 PhysLean.Relativity.Tensors.OverColor.Lift

[PhysLean/Relativity/Tensors/OverColor/Lift.lean](#) on GitHub.

lemma discreteFunctorMapEqIso_eq_action [\[source\]](#)

lemma discreteFunctorMapEqIso_eq_action {c1 c2 : Discrete C} (h : c1.as = c2.as) :
discreteFunctorMapEqIso F h = ((Action.forget _ _).mapIso
(F.mapIso (Discrete.eqToIso (**by** simp [h])))).toLinearEquiv

lemma objObj'_p_from_fin0 [\[source\]](#)

lemma objObj'_p_from_fin0 (c : Fin 0 → C) (g : G) :
(objObj' F (OverColor.mk c)).p g = LinearMap.id

2.35 PhysLean.Relativity.Tensors.TensorSpecies.Basic

[PhysLean/Relativity/Tensors/TensorSpecies/Basic.lean](#) on GitHub.

lemma τ_τ _apply [\[source\]](#)

@[simp]

lemma τ_τ _apply (c : S.C) : S. τ (S. τ c) = c

lemma contr_congr [\[source\]](#)

lemma contr_congr (c c' : S.C) (h : c = c') (x : S.FD.obj (Discrete.mk c))
(y : S.FD.obj (Discrete.mk (S. τ c))) :
(S.contr.app { as := c }).hom (x $\otimes_t$ [k] y) =
(S.contr.app { as := c' }).hom
((S.FD.map (Discrete.eqToHom (**by** simp [h])))).hom x $\otimes_t$

```
(S.FD.map (Discrete.eqToHom (by simp [h]))).hom y)
```

def numIndices

[\[source\]](#)

The number of indices n from a tensor.

```
@[nolint unusedArguments]
def numIndices {S : TensorSpecies k} {n : ℕ} {c : Fin n → S.C}
  (_ : S.F.obj (OverColor.mk c)) : ℕ
```

2.36 PhysLean.Relativity.Tensors.TensorSpecies.Basis

[PhysLean/Relativity/Tensors/TensorSpecies/Basis.lean](#) on GitHub.

def liftToContrSection

[\[source\]](#)

Given a b in $\Pi k, \text{Fin } (S.\text{repDim } ((c \circ i.\text{succAbove} \circ j.\text{succAbove}) k))$ and an x in $\text{Fin } (S.\text{repDim } (c i)) \times \text{Fin } (S.\text{repDim } (c (i.\text{succAbove } j)))$, the corresponding coordinate parameter in $(\Pi k, \text{Fin } (S.\text{repDim } (c k)))$.

```
def liftToContrSection {n : ℕ} {c : Fin n.succ.succ → S.C}
  {i : Fin n.succ.succ} {j : Fin n.succ}
  (b :  $\Pi k, \text{Fin } (S.\text{repDim } ((c \circ i.\text{succAbove} \circ j.\text{succAbove}) k))$ )
  (x :  $\text{Fin } (S.\text{repDim } (c i)) \times \text{Fin } (S.\text{repDim } (c (i.\text{succAbove } j)))$ ) :
  ( $\Pi k, \text{Fin } (S.\text{repDim } (c k))$ )
```

lemma liftToContrSection_mem_ContrSection

[\[source\]](#)

```
lemma liftToContrSection_mem_ContrSection {n : ℕ} {c : Fin n.succ.succ → S.C}
  {i : Fin n.succ.succ} {j : Fin n.succ}
  (b :  $\Pi k, \text{Fin } (S.\text{repDim } ((c \circ i.\text{succAbove} \circ j.\text{succAbove}) k))$ )
  (x :  $\text{Fin } (S.\text{repDim } (c i)) \times \text{Fin } (S.\text{repDim } (c (i.\text{succAbove } j)))$ ) :
  liftToContrSection b x ∈ ContrSection b
```

lemma liftToContrSection_apply_self_fst

[\[source\]](#)

```
@[simp]
lemma liftToContrSection_apply_self_fst {n : ℕ} {c : Fin n.succ.succ → S.C}
  {i : Fin n.succ.succ} {j : Fin n.succ}
  (b :  $\Pi k, \text{Fin } (S.\text{repDim } ((c \circ i.\text{succAbove} \circ j.\text{succAbove}) k))$ )
  (x :  $\text{Fin } (S.\text{repDim } (c i)) \times \text{Fin } (S.\text{repDim } (c (i.\text{succAbove } j)))$ ) :
  liftToContrSection b x i = x.1
```

lemma liftToContrSection_apply_self_snd

[\[source\]](#)

```
@[simp]
lemma liftToContrSection_apply_self_snd {n : ℕ} {c : Fin n.succ.succ → S.C}
  {i : Fin n.succ.succ} {j : Fin n.succ}
  (b :  $\Pi k, \text{Fin } (S.\text{repDim } ((c \circ i.\text{succAbove} \circ j.\text{succAbove}) k))$ )
  (x :  $\text{Fin } (S.\text{repDim } (c i)) \times \text{Fin } (S.\text{repDim } (c (i.\text{succAbove } j)))$ ) :
  liftToContrSection b x (i.succAbove j) = x.2
```

def contrSectionEquiv[\[source\]](#)

The equivalence between $\text{ContrSection } b$ and $\text{Fin } (S.\text{repDim } (c \ i)) \times \text{Fin } (S.\text{repDim } (c \ (i.\text{succAbove } j)))$.

```
def contrSectionEquiv {n : ℕ} {c : Fin n.succ.succ → S.C}
  {i : Fin n.succ.succ} {j : Fin n.succ}
  (b : Π k, Fin (S.repDim ((c ∘ i.succAbove ∘ j.succAbove) k))) :
  ContrSection b ≈ Fin (S.repDim (c i)) × Fin (S.repDim (c (i.succAbove j))) where
```

instance Fintype (ContrSection b)[\[source\]](#)

```
instance {n : ℕ} {c : Fin n.succ.succ → S.C}
  {i : Fin n.succ.succ} {j : Fin n.succ}
  (b : Π k, Fin (S.repDim ((c ∘ i.succAbove ∘ j.succAbove) k))) : Fintype (ContrSection b)
```

2.37 PhysLean.Relativity.Tensors.TensorSpecies.Tensor.Basic

[PhysLean/Relativity/Tensors/TensorSpecies/Tensor/Basic.lean](#) on GitHub.

abbrev Tensor[\[source\]](#)

The tensors associated with a list of indicies of a given color $c : \text{Fin } n \rightarrow S.C$. s

```
noncomputable abbrev Tensor {n : ℕ} (c : Fin n → S.C) : Type
```

abbrev ComponentIdx[\[source\]](#)

Given a list of indices $c : \text{Fin } n \rightarrow S.C$ e.g. $![.up, .down]$, the type $\text{ComponentIdx } c$ is the type of components indexes of a tensor with those indices e.g. $\langle 0, 2 \rangle$ corresponding to $T^0{}_2$.

```
abbrev ComponentIdx {n : ℕ} (c : Fin n → S.C) : Type
```

lemma ComponentIdx.congr_right[\[source\]](#)

```
lemma ComponentIdx.congr_right {n : ℕ} {c : Fin n → S.C} (b : ComponentIdx c)
  (i j : Fin n) (h : i = j) : b i = Fin.cast (by simp [h]) (b j)
```

abbrev Pure[\[source\]](#)

The type of pure tensors associated to a list of indices $c : \text{OverColor } S.C$. A pure tensor is a tensor which can be written in the form $v_1 \otimes_t v_2 \otimes_t v_3 \dots$.

```
abbrev Pure (S : TensorSpecies k) (c : Fin n → S.C) : Type
```

lemma Pure.congr_right[\[source\]](#)

```
@[simp]
lemma congr_right {n : ℕ} {c : Fin n → S.C} (p : Pure S c)
  (i j : Fin n) (h : i = j) : S.FD.map (eqToHom (by rw [h])) (p j) = p i
```

lemma congr_mid[\[source\]](#)

```

lemma congr_mid {n : ℕ} {c : Fin n → S.C} (c' : S.C) (p : Pure S c)
  (i j : Fin n) (h : i = j) (hi : c i = c') (hj : c j = c') :
  S.FD.map (eqToHom (by rw [hi] : { as := c i } = { as := c' })) (p i) =
  S.FD.map (eqToHom (by rw [hj] : { as := c j } = { as := c' })) (p j)

```

lemma map_map_apply[\[source\]](#)

```

lemma map_map_apply {n : ℕ} {c : Fin n → S.C} (c1 c2 : S.C) (p : Pure S c) (i : Fin n)
  (f : ({ as := c i } : Discrete S.C) → { as := c1 })
  (g : ({ as := c1 } : Discrete S.C) → { as := c2 }) :
  (ConcreteCategory.hom (S.FD.map g))
  ((ConcreteCategory.hom (S.FD.map f)) (p i)) =
  S.FD.map (f >> g) (p i)

```

def toTensor[\[source\]](#)

The tensor corresponding to a pure tensor.

```

noncomputable def toTensor {n : ℕ} {c : Fin n → S.C} (p : Pure S c) : S.Tensor c

```

lemma toTensor_apply[\[source\]](#)

```

lemma toTensor_apply {n : ℕ} (c : Fin n → S.C) (p : Pure S c) :
  toTensor p = PiTensorProduct.tprod k p

```

def update[\[source\]](#)

Given a list of indices c of n indices, a pure tensor p , an element $i : \text{Fin } n$ and a x in $S.FD.obj$ ($\text{Discrete.mk } (c \ i)$) then update $p \ i \ x$ corresponds to p where the i th part of p is replaced with x .

E.g. if $n = 2$ and $p = v_0 \otimes_t v_1$ then update $p \ 0 \ x = x \otimes_t v_1$.

```

def update {n : ℕ} {c : Fin n → S.C} [inst : DecidableEq (Fin n)] (p : Pure S c) (i : Fin n)
  (x : S.FD.obj (Discrete.mk (c i))) : Pure S c

```

lemma update_same[\[source\]](#)

```

@[simp]
lemma update_same {n : ℕ} {c : Fin n → S.C} [inst : DecidableEq (Fin n)] (p : Pure S c) (i :
  Fin n)
  (x : S.FD.obj (Discrete.mk (c i))) : (update p i x) i = x

```

lemma update_succAbove_apply[\[source\]](#)

```

@[simp]
lemma update_succAbove_apply {n : ℕ} {c : Fin (n + 1) → S.C} [inst : DecidableEq (Fin (n + 1))]
  (p : Pure S c) (i : Fin (n + 1)) (j : Fin n) (x : S.FD.obj (Discrete.mk (c (i.succAbove j))
  )) :
  update p (i.succAbove j) x i = p i

```

lemma toTensor_update_add[\[source\]](#)

```
@[simp]
lemma toTensor_update_add {n : ℕ} {c : Fin n → S.C} [inst : DecidableEq (Fin n)] (p : Pure S c)
  (i : Fin n) (x y : S.FD.obj (Discrete.mk (c i))) :
  (update p i (x + y)).toTensor = (update p i x).toTensor + (update p i y).toTensor
```

lemma toTensor_update_smul[\[source\]](#)

```
@[simp]
lemma toTensor_update_smul {n : ℕ} {c : Fin n → S.C} [inst : DecidableEq (Fin n)] (p : Pure S c)
  (i : Fin n) (r : k) (y : S.FD.obj (Discrete.mk (c i))) :
  (update p i (r • y)).toTensor = r • (update p i y).toTensor
```

def drop[\[source\]](#)

Given a list of indices c of length $n + 1$, a pure tensor p and an $i : \text{Fin } (n + 1)$, then $\text{drop } p \ i$ is the tensor p with it's i th part dropped.

For example, if $n = 2$ and $p = v_0 \otimes_t v_1 \otimes_t v_2$ then $\text{drop } p \ 1 = v_0 \otimes_t v_2$.

```
def drop {n : ℕ} {c : Fin (n + 1) → S.C} (p : Pure S c) (i : Fin (n + 1)) :
  Pure S (c ∘ i.succAbove)
```

lemma update_succAbove_drop[\[source\]](#)

```
@[simp]
lemma update_succAbove_drop {n : ℕ} {c : Fin (n + 1) → S.C} [inst : DecidableEq (Fin (n + 1))]
  (p : Pure S c) (i : Fin (n + 1)) (k : Fin n) (x : S.FD.obj (Discrete.mk (c (i.succAbove k)))) :
  (update p (i.succAbove k) x).drop i = (p.drop i).update k x
```

lemma update_drop_self[\[source\]](#)

```
@[simp]
lemma update_drop_self {n : ℕ} {c : Fin (n + 1) → S.C} [inst : DecidableEq (Fin (n + 1))]
  (p : Pure S c) (i : Fin (n + 1)) (x : S.FD.obj (Discrete.mk (c i))) :
  (update p i x).drop i = p.drop i
```

lemma μ_toTensor_tmul_toTensor[\[source\]](#)

```
lemma μ_toTensor_tmul_toTensor {n1 n2} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C}
  (t : Pure S c) (t1 : Pure S c1) :
  ((Functor.LaxMonoidal.μ S.F _ _).hom (t.toTensor ⊗_t t1.toTensor)) =
  PiTensorProduct.tprod k (fun | Sum.inl i => t i | Sum.inr i => t1 i)
```

def component[\[source\]](#)

Given an element b of $\text{ComponentIdx } c$ and a pure tensor p then $\text{component } p \ b$ is the element of the ring k corresponding to the component of p in the direction b .

For example, if $p = v \otimes_t w$ and $b = \langle 0, 1 \rangle$ then $\text{component } p \ b = v^0 \otimes_t w^1$.

```
def component {n : ℕ} {c : Fin n → S.C} (p : Pure S c) (b : ComponentIdx c) : k
```

lemma component_eq[\[source\]](#)

```
lemma component_eq {n : ℕ} {c : Fin n → S.C} (p : Pure S c) (b : ComponentIdx c) :
  p.component b = ∏ i, (S.basis (c i)).repr (p i) (b i)
```

lemma component_eq_drop[\[source\]](#)

```
lemma component_eq_drop {n : ℕ} {c : Fin (n + 1) → S.C} (p : Pure S c) (i : Fin (n + 1))
  (b : ComponentIdx c) :
  p.component b = ((S.basis (c i)).repr (p i) (b i)) *
  ((drop p i).component (fun j => b (i.succAbove j)))
```

lemma component_update_add[\[source\]](#)

```
@[simp]
lemma component_update_add {n : ℕ} [inst : DecidableEq (Fin n)]
  {c : Fin n → S.C} (p : Pure S c) (i : Fin n)
  (x y : S.FD.obj (Discrete.mk (c i))) (b : ComponentIdx c) :
  (update p i (x + y)).component b = (update p i x).component b +
  (update p i y).component b
```

lemma component_update_smul[\[source\]](#)

```
@[simp]
lemma component_update_smul {n : ℕ} [inst : DecidableEq (Fin n)]
  {c : Fin n → S.C} (p : Pure S c) (i : Fin n)
  (x : k) (y : S.FD.obj (Discrete.mk (c i))) (b : ComponentIdx c) :
  (update p i (x • y)).component b = x * (update p i y).component b
```

def Pure.componentMap[\[source\]](#)

The multilinear map taking pure tensors p to a map $\text{ComponentIdx } c \rightarrow k$ which when evaluated returns the components of p .

```
noncomputable def componentMap {n : ℕ} (c : Fin n → S.C) :
  MultilinearMap k (fun i => S.FD.obj (Discrete.mk (c i))) (ComponentIdx c → k) where
```

lemma componentMap_apply[\[source\]](#)

```
@[simp]
lemma componentMap_apply {n : ℕ} (c : Fin n → S.C)
  (p : Pure S c) : componentMap c p = p.component
```

def basisVector[\[source\]](#)

Given an component idx b in $\text{ComponentIdx } c$, $\text{basisVector } c \ b$ is the pure tensor formed by $S.\text{basis } (c \ i) \ (b \ i)$.

```
noncomputable def basisVector {n : ℕ} (c : Fin n → S.C) (b : ComponentIdx c) : Pure S c
```

lemma component_basisVector[\[source\]](#)

```
@[simp]
lemma component_basisVector {n : ℕ} (c : Fin n → S.C) (b1 b2 : ComponentIdx c) :
  (basisVector c b1).component b2 = if b1 = b2 then 1 else 0
```

lemma induction_on_pure[\[source\]](#)

```
lemma induction_on_pure {n : ℕ} {c : Fin n → S.C} {P : S.Tensor c → Prop}
  (h : ∀ (p : Pure S c), P p.toTensor)
  (hsmul : ∀ (r : k) t, P t → P (r • t))
  (hadd : ∀ t1 t2, P t1 → P t2 → P (t1 + t2)) (t : S.Tensor c) : P t
```

def Tensor.componentMap[\[source\]](#)

The linear map from tensors to its components.

```
def componentMap {n : ℕ} (c : Fin n → S.C) : S.Tensor c →l[k] (ComponentIdx c → k)
```

lemma componentMap_pure[\[source\]](#)

```
@[simp]
lemma componentMap_pure {n : ℕ} (c : Fin n → S.C)
  (p : Pure S c) : componentMap c (p.toTensor) = Pure.componentMap c p
```

def ofComponents[\[source\]](#)

The tensor created from it's components.

```
def ofComponents {n : ℕ} (c : Fin n → S.C) :
  (ComponentIdx c → k) →l[k] S.Tensor c where
```

lemma componentMap_ofComponents[\[source\]](#)

```
@[simp]
lemma componentMap_ofComponents {n : ℕ} (c : Fin n → S.C) (f : ComponentIdx c → k) :
  componentMap c (ofComponents c f) = f
```

lemma ofComponents_componentMap[\[source\]](#)

```
@[simp]
lemma ofComponents_componentMap {n : ℕ} (c : Fin n → S.C) (t : S.Tensor c) :
  ofComponents c (componentMap c t) = t
```

def basis[\[source\]](#)

The basis of tensors.

```
def basis {n : ℕ} (c : Fin n → S.C) : Basis (ComponentIdx c) k (S.Tensor c) where
```

lemma basis_apply[\[source\]](#)

```
lemma basis_apply {n : ℕ} (c : Fin n → S.C) (b : ComponentIdx c) :
  basis c b = (Pure.basisVector c b).toTensor
```

lemma induction_on_basis[\[source\]](#)

```

lemma induction_on_basis {n : ℕ} {c : Fin n → S.C} {P : S.Tensor c → Prop}
  (h : ∀ b, P (basis c b)) (hzero : P 0)
  (hsmul : ∀ (r : k) t, P t → P (r • t))
  (hadd : ∀ t1 t2, P t1 → P t2 → P (t1 + t2)) (t : S.Tensor c) : P t

```

instance actionP[\[source\]](#)

```

noncomputable instance actionP : MulAction S.G (Pure S c) where

```

instance SMul (S.G) (Pure S c)[\[source\]](#)

```

noncomputable instance : SMul (S.G) (Pure S c)

```

lemma actionP_eq[\[source\]](#)

```

lemma actionP_eq {g : S.G} {p : Pure S c} : g • p = fun i => (S.FD.obj _).p g (p i)

```

lemma drop_actionP[\[source\]](#)

```

@[simp]
lemma drop_actionP {n : ℕ} {c : Fin (n + 1) → S.C} {i : Fin (n + 1)} {p : Pure S c} (g : S.G) :
  (g • p).drop i = g • (p.drop i)

```

instance actionT[\[source\]](#)

```

noncomputable instance actionT : MulAction S.G (S.Tensor c) where

```

lemma actionT_eq[\[source\]](#)

```

lemma actionT_eq {g : S.G} {t : S.Tensor c} : g • t = (S.F.obj (0verColor.mk c)).p g t

```

lemma actionT_pure[\[source\]](#)

```

lemma actionT_pure {g : S.G} {p : Pure S c} :
  g • p.toTensor = Pure.toTensor (g • p)

```

lemma actionT_add[\[source\]](#)

```

@[simp]
lemma actionT_add {g : S.G} {t1 t2 : S.Tensor c} :
  g • (t1 + t2) = g • t1 + g • t2

```

lemma actionT_smul[\[source\]](#)

```

@[simp]
lemma actionT_smul {g : S.G} {r : k} {t : S.Tensor c} :
  g • (r • t) = r • (g • t)

```

lemma actionT_zero[\[source\]](#)

```
@[simp]
lemma actionT_zero {g : S.G} : g • (0 : S.Tensor c) = 0
```

def PermCond[\[source\]](#)

Given two lists of indices $c : \text{Fin } n \rightarrow S.C$ and $c1 : \text{Fin } m \rightarrow S.C$ a map $\sigma : \text{Fin } m \rightarrow \text{Fin } n$ satisfies the condition $\text{PermCond } c \ c1 \ \sigma$ if it is: - A bijection - Forms a commutative triangle with c and $c1$.

```
def PermCond {n m : ℕ} (c : Fin n → S.C) (c1 : Fin m → S.C)
  (σ : Fin m → Fin n) : Prop
```

def inv[\[source\]](#)

For a map σ satisfying $\text{PermCond } c \ c1 \ \sigma$, the inverse of that map.

```
def PermCond.inv {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (σ : Fin m → Fin n) (h : PermCond c c1 σ) : Fin n → Fin m
```

def toEquiv[\[source\]](#)

For a map $\sigma : \text{Fin } m \rightarrow \text{Fin } n$ satisfying $\text{PermCond } c \ c1 \ \sigma$, that map lifted to an equivalence between $\text{Fin } n$ and $\text{Fin } m$.

```
def PermCond.toEquiv {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  {σ : Fin m → Fin n} (h : PermCond c c1 σ) :
  Fin n ≈ Fin m where
```

lemma preserve_color[\[source\]](#)

```
lemma PermCond.preserve_color {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  {σ : Fin m → Fin n} (h : PermCond c c1 σ) :
  ∀ (x : Fin m), c1 x = (c ∘ σ) x
```

def toHom[\[source\]](#)

For a map $\sigma : \text{Fin } m \rightarrow \text{Fin } n$ satisfying $\text{PermCond } c \ c1 \ \sigma$, that map lifted to a morphism in the $\text{OverColor } C$ category.

```
def PermCond.toHom {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  {σ : Fin m → Fin n} (h : PermCond c c1 σ) :
  OverColor.mk c → OverColor.mk c1
```

lemma ofHom[\[source\]](#)

Given a morphism in the $\text{OverColor } C$ between c and $c1$ category the corresponding morphism $(\text{Hom.toEquiv } \sigma).\text{symm}$ satisfies the PermCond .

```
lemma PermCond.ofHom {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (σ : OverColor.mk c → OverColor.mk c1) :
  PermCond c c1 (Hom.toEquiv σ).symm
```

lemma comp[\[source\]](#)

The composition of two maps satisfying PermCond also satisfies the PermCond.

```
lemma PermCond.comp {n n1 n2 : ℕ} {c : Fin n → S.C} {c1 : Fin n1 → S.C}
  {c2 : Fin n2 → S.C} {σ : Fin n1 → Fin n} {σ2 : Fin n2 → Fin n1}
  (h : PermCond c c1 σ) (h2 : PermCond c1 c2 σ2) :
  PermCond c c2 (σ ∘ σ2)
```

lemma fin_cast_permCond[\[source\]](#)

```
lemma fin_cast_permCond (n n1 : ℕ) {c : Fin n → S.C} (h : n1 = n) :
  PermCond c (c ∘ Fin.cast h) (Fin.cast h)
```

def permP[\[source\]](#)

Given a permutation $\sigma : \text{Fin } m \rightarrow \text{Fin } n$ of indices satisfying PermCond through h , and a pure tensor p , $\text{permP } \sigma \ h \ p$ is the pure tensor permuted according to σ .

For example if $m = n = 2$ and $\sigma = ![1, 0]$, and $p = v \otimes_t w$ then $\text{permP } \sigma \ _ \ p = w \otimes_t v$.

```
def Pure.permP {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (σ : Fin m → Fin n) (h : PermCond c c1 σ) (p : Pure S c) : Pure S c1
```

lemma permP_basisVector[\[source\]](#)

@[simp]

```
lemma Pure.permP_basisVector {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (σ : Fin m → Fin n) (h : PermCond c c1 σ) (b : ComponentIdx c) :
  Pure.permP σ h (Pure.basisVector c b) =
  Pure.basisVector c1 (fun i => Fin.cast (by simp [h.preserve_color]) (b (σ i)))
```

def permT[\[source\]](#)

Given a permutation $\sigma : \text{Fin } m \rightarrow \text{Fin } n$ of indices satisfying PermCond through h , and a tensor t , $\text{permT } \sigma \ h \ t$ is the tensor tensor permuted according to σ .

```
noncomputable def permT {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (σ : Fin m → Fin n) (h : PermCond c c1 σ) : S.Tensor c →l[k] S.Tensor c1 where
```

lemma permT_pure[\[source\]](#)

```
lemma permT_pure {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  {σ : Fin m → Fin n} (h : PermCond c c1 σ) (p : Pure S c) :
  permT σ h p.toTensor = (p.permP σ h).toTensor
```

lemma permT_equivariant[\[source\]](#)

@[simp]

```
lemma permT_equivariant {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  {σ : Fin m → Fin n} (h : PermCond c c1 σ) (g : S.G) (t : S.Tensor c) :
  permT σ h (g • t) = g • permT σ h t
```

lemma permP_congr[\[source\]](#)

@[congr]

```

lemma Pure.permP_congr {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  {σ σ1 : Fin m → Fin n} {h : PermCond c c1 σ} {h1 : PermCond c c1 σ1}
  {p p1 : Pure S c} (hmap : σ = σ1) (hpure : p = p1) :
  Pure.permP σ h p = Pure.permP σ1 h1 p1

```

lemma permT_congr[\[source\]](#)

@[congr]

```

lemma permT_congr {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  {σ σ1 : Fin m → Fin n} {h : PermCond c c1 σ} {h1 : PermCond c c1 σ1}
  (hmap : σ = σ1) {t t1 : S.Tensor c} (htensor : t = t1) :
  permT σ h t = permT σ1 h1 t1

```

lemma permP_permP[\[source\]](#)

@[simp]

```

lemma Pure.permP_permP {n m1 m2 : ℕ} {c : Fin n → S.C} {c1 : Fin m1 → S.C} {c2 : Fin m2 → S.C}
  {σ : Fin m1 → Fin n} {σ2 : Fin m2 → Fin m1} (h : PermCond c c1 σ) (h2 : PermCond c1 c2 σ2)
  (p : Pure S c) :
  Pure.permP σ2 h2 (Pure.permP σ h p) = Pure.permP (σ ∘ σ2) (PermCond.comp h h2) p

```

lemma permT_permT[\[source\]](#)

@[simp]

```

lemma permT_permT {n m1 m2 : ℕ} {c : Fin n → S.C} {c1 : Fin m1 → S.C} {c2 : Fin m2 → S.C}
  {σ : Fin m1 → Fin n} {σ2 : Fin m2 → Fin m1} (h : PermCond c c1 σ) (h2 : PermCond c1 c2 σ2)
  (t : S.Tensor c) :
  permT σ2 h2 (permT σ h t) = permT (σ ∘ σ2) (PermCond.comp h h2) t

```

def ComponentIdx.prodEquiv[\[source\]](#)

The equivalence between ComponentIdx (Sum.elim c c1 ∘ ↑finSumFinEquiv.symm) and $\prod (i : \text{Fin } n1 \otimes \text{Fin } n2), \text{Fin } (S.\text{repDim } (\text{Sum.elim } c \text{ c1 } i))$.

```

def ComponentIdx.prodEquiv {n1 n2 : ℕ} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C} :
  ComponentIdx (Sum.elim c c1 ∘ ↑finSumFinEquiv.symm) =
   $\prod (i : \text{Fin } n1 \otimes \text{Fin } n2), \text{Fin } (S.\text{repDim } (\text{Sum.elim } c \text{ c1 } i))$ 

```

def prod[\[source\]](#)

The product of two component indices.

```

def ComponentIdx.prod {n1 n2 : ℕ} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C}
  (b : ComponentIdx c) (b1 : ComponentIdx c1) :
  ComponentIdx (Sum.elim c c1 ∘ ↑finSumFinEquiv.symm)

```

lemma prod_apply_finSumFinEquiv[\[source\]](#)

```

lemma ComponentIdx.prod_apply_finSumFinEquiv {n1 n2 : ℕ} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C}
  (b : ComponentIdx c) (b1 : ComponentIdx c1) (i : Fin n1 ⊗ Fin n2) :
  ComponentIdx.prod b b1 (finSumFinEquiv i) =

```

```

match i with
| Sum.inl i => Fin.cast (by simp) (b i)
| Sum.inr i => Fin.cast (by simp) (b1 i)

```

def Pure.prodEquiv[\[source\]](#)

The equivalence between pure tensors based on a product of lists of indices, and the type $\Pi (i : \text{Fin } n1 \otimes \text{Fin } n2), S.FD.obj (\text{Discrete.mk } ((\text{Sum.elim } c \ c1) \ i))$.

```

def Pure.prodEquiv {n1 n2 : ℕ} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C} :
  Pure S (Sum.elim c c1 ∘ ↑finSumFinEquiv.symm) ≈
  Π (i : Fin n1 ⊗ Fin n2), S.FD.obj (Discrete.mk ((Sum.elim c c1) i))

```

def prodP[\[source\]](#)

Given two pure tensors $p1 : \text{Pure } S \ c$ and $p2 : \text{Pure } S \ c$, $\text{prodP } p \ p2$ is the tensor product of those tensors returning an element in $\text{Pure } S \ c$ ($\text{Sum.elim } c \ c1 \circ \uparrow\text{finSumFinEquiv.symm}$).

```

def Pure.prodP {n1 n2} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C}
  (p1 : Pure S c) (p2 : Pure S c1) : Pure S (Sum.elim c c1 ∘ ↑finSumFinEquiv.symm)

```

lemma prodP_apply_finSumFinEquiv[\[source\]](#)

```

lemma Pure.prodP_apply_finSumFinEquiv {n1 n2} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C}
  (p1 : Pure S c) (p2 : Pure S c1) (i : Fin n1 ⊗ Fin n2) :
  Pure.prodP p1 p2 (finSumFinEquiv i) =
  match i with
  | Sum.inl i => S.FD.map (eqToHom (by simp)) (p1 i)
  | Sum.inr i => S.FD.map (eqToHom (by simp)) (p2 i)

```

lemma prodP_apply_castAdd[\[source\]](#)

```

@[simp]
lemma Pure.prodP_apply_castAdd {n1 n2} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C}
  (p1 : Pure S c) (p2 : Pure S c1) (i : Fin n1) :
  Pure.prodP p1 p2 (Fin.castAdd n2 i) =
  S.FD.map (eqToHom (by simp)) (p1 i)

```

lemma prodP_apply_natAdd[\[source\]](#)

```

@[simp]
lemma Pure.prodP_apply_natAdd {n1 n2} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C}
  (p1 : Pure S c) (p2 : Pure S c1) (i : Fin n2) :
  Pure.prodP p1 p2 (Fin.natAdd n1 i) =
  S.FD.map (eqToHom (by simp)) (p2 i)

```

lemma prodP_basisVector[\[source\]](#)

```

lemma Pure.prodP_basisVector {n n1 : ℕ} {c : Fin n → S.C} {c1 : Fin n1 → S.C}
  (b : ComponentIdx c) (b1 : ComponentIdx c1) :
  Pure.prodP (Pure.basisVector c b) (Pure.basisVector c1 b1) =
  Pure.basisVector _ (b.prod b1)

```

def Tensor.prodEquiv[\[source\]](#)

The equivalence between the type $S.F.obj$ ($OverColor.mk$ ($Sum.elim$ c $c1$)) and the type $S.Tensor$ ($Sum.elim$ c $c1 \circ \uparrow finSumFinEquiv.symm$).

```
noncomputable def prodEquiv {n1 n2} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C} :
  S.F.obj (OverColor.mk (Sum.elim c c1)) ≈l[k] S.Tensor (Sum.elim c c1 ∘ ↑finSumFinEquiv.symm)
```

lemma prodEquiv_symm_pure[\[source\]](#)

```
lemma prodEquiv_symm_pure {n1 n2} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C}
  (p : Pure S (Sum.elim c c1 ∘ ↑finSumFinEquiv.symm)) :
  prodEquiv.symm p.toTensor = PiTensorProduct.tprod k (Pure.prodEquiv p)
```

def prodT[\[source\]](#)

The tensor product of two tensors as a bi-linear map from $S.Tensor$ c and $S.Tensor$ $c1$ to $S.Tensor$ ($Sum.elim$ c $c1 \circ \uparrow finSumFinEquiv.symm$).

```
noncomputable def prodT {n1 n2} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C} :
  S.Tensor c →l[k] S.Tensor c1 →l[k] S.Tensor (Sum.elim c c1 ∘ ↑finSumFinEquiv.symm)
```

lemma prodT_pure[\[source\]](#)

```
lemma prodT_pure {n1 n2} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C}
  (t : Pure S c) (t1 : Pure S c1) :
  (t.toTensor).prodT (t1.toTensor) = (Pure.prodP t t1).toTensor
```

def prodLeftMap[\[source\]](#)

Given a map $\sigma : Fin\ n \rightarrow Fin\ n'$, the induced map $Fin\ (n + n2) \rightarrow Fin\ (n' + n2)$.

```
def prodLeftMap (n2 : ℕ) (σ : Fin n → Fin n') : Fin (n + n2) → Fin (n' + n2)
```

def prodRightMap[\[source\]](#)

Given a map $\sigma : Fin\ n \rightarrow Fin\ n'$, the induced map $Fin\ (n2 + n) \rightarrow Fin\ (n2 + n')$.

```
def prodRightMap (n2 : ℕ) (σ : Fin n → Fin n') : Fin (n2 + n) → Fin (n2 + n')
```

def prodAssocMap[\[source\]](#)

The map between $Fin\ (n1 + n2 + n3)$ and $Fin\ (n1 + (n2 + n3))$ formed by casting.

```
def prodAssocMap (n1 n2 n3 : ℕ) : Fin (n1 + n2 + n3) → Fin (n1 + (n2 + n3))
```

lemma prodAssocMap_castAdd_castAdd[\[source\]](#)

```
@[simp]
lemma prodAssocMap_castAdd_castAdd {n1 n2 n3 : ℕ} (i : Fin n1) :
  prodAssocMap n1 n2 n3 (Fin.castAdd n3 (Fin.castAdd n2 i)) =
  finSumFinEquiv (Sum.inl i)
```

lemma prodAssocMap_castAdd_natAdd[\[source\]](#)

```
@[simp]
lemma prodAssocMap_castAdd_natAdd {n1 n2 n3 : ℕ} (i : Fin n2) :
  prodAssocMap n1 n2 n3 (Fin.castAdd n3 (Fin.natAdd n1 i)) =
  finSumFinEquiv (Sum.inr (finSumFinEquiv (Sum.inl i)))
```

lemma prodAssocMap_natAdd[\[source\]](#)

```
@[simp]
lemma prodAssocMap_natAdd {n1 n2 n3 : ℕ} (i : Fin (n3)) :
  prodAssocMap n1 n2 n3 (Fin.natAdd (n1 + n2) i) =
  finSumFinEquiv (Sum.inr (finSumFinEquiv (Sum.inr i)))
```

def prodAssocMap'[\[source\]](#)

The map between $\text{Fin } (n1 + (n2 + n3))$ and $\text{Fin } (n1 + n2 + n3)$ formed by casting.

```
def prodAssocMap' (n1 n2 n3 : ℕ) : Fin (n1 + (n2 + n3)) → Fin (n1 + n2 + n3)
```

lemma prodAssocMap'_castAdd[\[source\]](#)

```
@[simp]
lemma prodAssocMap'_castAdd {n1 n2 n3 : ℕ} (i : Fin n1) :
  prodAssocMap' n1 n2 n3 (Fin.castAdd (n2 + n3) i) =
  finSumFinEquiv (Sum.inl (finSumFinEquiv (Sum.inl i)))
```

lemma prodAssocMap'_natAdd_castAdd[\[source\]](#)

```
@[simp]
lemma prodAssocMap'_natAdd_castAdd {n1 n2 n3 : ℕ} (i : Fin n2) :
  prodAssocMap' n1 n2 n3 (Fin.natAdd n1 (Fin.castAdd n3 i)) =
  finSumFinEquiv (Sum.inl (finSumFinEquiv (Sum.inr i)))
```

lemma prodAssocMap'_natAdd_natAdd[\[source\]](#)

```
@[simp]
lemma prodAssocMap'_natAdd_natAdd {n1 n2 n3 : ℕ} (i : Fin n3) :
  prodAssocMap' n1 n2 n3 (Fin.natAdd n1 (Fin.natAdd n2 i)) =
  finSumFinEquiv (Sum.inr i)
```

def prodSwapMap[\[source\]](#)

The map between $\text{Fin } (n1 + n2)$ and $\text{Fin } (n2 + n1)$ formed by swapping elements.

```
def prodSwapMap (n1 n2 : ℕ) : Fin (n1 + n2) → Fin (n2 + n1)
```

lemma prodLeftMap_permCond[\[source\]](#)

```
@[simp]
lemma prodLeftMap_permCond {σ : Fin n' → Fin n} (c2 : Fin n2 → S.C) (h : PermCond c c' σ) :
  PermCond (Sum.elim c c2 ∘ finSumFinEquiv.symm)
  (Sum.elim c' c2 ∘ finSumFinEquiv.symm) (prodLeftMap n2 σ)
```

lemma prodRightMap_permCond[\[source\]](#)

```
@[simp]
lemma prodRightMap_permCond {σ : Fin n' → Fin n} (c2 : Fin n2 → S.C) (h : PermCond c c' σ) :
  PermCond (Sum.elim c2 c ◦ finSumFinEquiv.symm)
    (Sum.elim c2 c' ◦ finSumFinEquiv.symm) (prodRightMap n2 σ)
```

lemma prodSwapMap_permCond[\[source\]](#)

```
@[simp]
lemma prodSwapMap_permCond {n1 n2 : ℕ} {c : Fin n1 → S.C} {c2 : Fin n2 → S.C} :
  PermCond (Sum.elim c c2 ◦ finSumFinEquiv.symm)
    (Sum.elim c2 c ◦ finSumFinEquiv.symm) (prodSwapMap n2 n1)
```

lemma prodAssocMap_permCond[\[source\]](#)

```
@[simp]
lemma prodAssocMap_permCond {n1 n2 n3 : ℕ} {c : Fin n1 → S.C} {c2 : Fin n2 → S.C}
  {c3 : Fin n3 → S.C} :
  PermCond (Sum.elim c (Sum.elim c2 c3 ◦ finSumFinEquiv.symm) ◦ finSumFinEquiv.symm)
    (Sum.elim (Sum.elim c c2 ◦ finSumFinEquiv.symm) c3 ◦ finSumFinEquiv.symm)
    (prodAssocMap n1 n2 n3)
```

lemma prodAssocMap'_permCond[\[source\]](#)

```
@[simp]
lemma prodAssocMap'_permCond {n1 n2 n3 : ℕ} {c : Fin n1 → S.C} {c2 : Fin n2 → S.C}
  {c3 : Fin n3 → S.C} : PermCond
  (Sum.elim (Sum.elim c c2 ◦ finSumFinEquiv.symm) c3 ◦ finSumFinEquiv.symm)
  (Sum.elim c (Sum.elim c2 c3 ◦ finSumFinEquiv.symm) ◦ finSumFinEquiv.symm)
  (prodAssocMap' n1 n2 n3)
```

lemma prodP_equivariant[\[source\]](#)

```
@[simp]
lemma Pure.prodP_equivariant {n1 n2} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C}
  (g : S.G) (p : Pure S c) (p1 : Pure S c1) :
  prodP (g • p) (g • p1) = g • prodP p p1
```

lemma prodT_equivariant[\[source\]](#)

```
@[simp]
lemma prodT_equivariant {n1 n2} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C}
  (g : S.G) (t : S.Tensor c) (t1 : S.Tensor c1) :
  prodT (g • t) (g • t1) = g • prodT t t1
```

lemma prodP_swap[\[source\]](#)

```
lemma Pure.prodP_swap {n n1} {c : Fin n → S.C}
  {c1 : Fin n1 → S.C}
  (p : Pure S c) (p1 : Pure S c1) :
  Pure.prodP p p1 = permP (prodSwapMap n n1) (prodSwapMap_permCond) (Pure.prodP p1 p)
```

lemma prodT_swap[\[source\]](#)

```

lemma prodT_swap {n n1} {c : Fin n → S.C}
  {c1 : Fin n1 → S.C}
  (t : S.Tensor c) (t1 : S.Tensor c1) :
  prodT t t1 = permT (prodSwapMap n n1) (prodSwapMap_permCond) (prodT t1 t)

```

lemma prodP_permP_left[\[source\]](#)

```

@[simp]
lemma Pure.prodP_permP_left {n n'} {c : Fin n → S.C} {c' : Fin n' → S.C}
  (σ : Fin n' → Fin n) (h : PermCond c c' σ) (p : Pure S c) (p2 : Pure S c2) :
  Pure.prodP (permP σ h p) p2 = permP (prodLeftMap n2 σ)
    (prodLeftMap_permCond c2 h) (Pure.prodP p p2)

```

lemma prodT_permT_left[\[source\]](#)

```

@[simp]
lemma prodT_permT_left {n n'} {c : Fin n → S.C} {c' : Fin n' → S.C}
  (σ : Fin n' → Fin n) (h : PermCond c c' σ) (t : S.Tensor c) (t2 : S.Tensor c2) :
  prodT (permT σ h t) t2 = permT (prodLeftMap n2 σ) (prodLeftMap_permCond c2 h) (prodT t t2)

```

lemma prodP_permP_right[\[source\]](#)

```

@[simp]
lemma Pure.prodP_permP_right {n n'} {c : Fin n → S.C} {c' : Fin n' → S.C}
  (σ : Fin n' → Fin n) (h : PermCond c c' σ) (p : Pure S c) (p2 : Pure S c2) :
  prodP p2 (permP σ h p) = permP (prodRightMap n2 σ)
    (prodRightMap_permCond c2 h) (Pure.prodP p2 p)

```

lemma prodT_permT_right[\[source\]](#)

```

@[simp]
lemma prodT_permT_right {n n'} {c : Fin n → S.C} {c' : Fin n' → S.C}
  (σ : Fin n' → Fin n) (h : PermCond c c' σ) (t : S.Tensor c) (t2 : S.Tensor c2) :
  prodT t2 (permT σ h t) = permT (prodRightMap n2 σ)
    (prodRightMap_permCond c2 h) (prodT t2 t)

```

lemma prodP_assoc[\[source\]](#)

```

lemma Pure.prodP_assoc {n n1 n2} {c : Fin n → S.C}
  {c1 : Fin n1 → S.C} {c2 : Fin n2 → S.C}
  (p : Pure S c) (p1 : Pure S c1) (p2 : Pure S c2) :
  prodP (Pure.prodP p p1) p2 =
  permP (prodAssocMap n n1 n2) (prodAssocMap_permCond) (Pure.prodP p (Pure.prodP p1 p2))

```

lemma prodT_assoc[\[source\]](#)

```

lemma prodT_assoc {n n1 n2} {c : Fin n → S.C}
  {c1 : Fin n1 → S.C} {c2 : Fin n2 → S.C}
  (t : S.Tensor c) (t1 : S.Tensor c1) (t2 : S.Tensor c2) :
  prodT (prodT t t1) t2 =
  permT (prodAssocMap n n1 n2) (prodAssocMap_permCond) (prodT t (prodT t1 t2))

```

lemma prodP_assoc'[\[source\]](#)

```

lemma Pure.prodP_assoc' {n n1 n2} {c : Fin n → S.C}
  {c1 : Fin n1 → S.C} {c2 : Fin n2 → S.C}
  (p : Pure S c) (p1 : Pure S c1) (p2 : Pure S c2) :
  prodP p (prodP p1 p2) =
  permP (prodAssocMap' n n1 n2) (prodAssocMap'_permCond) (prodP (prodP p p1) p2)

```

lemma prodT_assoc'[\[source\]](#)

```

lemma prodT_assoc' {n n1 n2} {c : Fin n → S.C}
  {c1 : Fin n1 → S.C} {c2 : Fin n2 → S.C}
  (t : S.Tensor c) (t1 : S.Tensor c1) (t2 : S.Tensor c2) :
  prodT t (prodT t1 t2) =
  permT (prodAssocMap' n n1 n2) (prodAssocMap'_permCond) (prodT (prodT t t1) t2)

```

2.38 PhysLean.Relativity.Tensors.TensorSpecies.Tensor.Contraction

[PhysLean/Relativity/Tensors/TensorSpecies/Tensor/Contraction.lean](#) on GitHub.

def dropPairEmb[\[source\]](#)

The embedding of $\text{Fin } n$ into $\text{Fin } (n + 1 + 1)$ with a hole at i and j .

```

def dropPairEmb (i j : Fin (n + 1 + 1)) (hij : i ≠ j) : Fin n ↪ Fin (n + 1 + 1)

```

lemma dropPairEmb_apply_eq_orderIsoOfFin[\[source\]](#)

```

lemma dropPairEmb_apply_eq_orderIsoOfFin {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (m : Fin n) :
  (dropPairEmb i j hij) m = (Finset.orderIsoOfFin {i, j})c
  (by rw [Finset.card_compl]; simp [Finset.card_pair hij])) m

```

lemma dropPairEmb_symm[\[source\]](#)

```

lemma dropPairEmb_symm (i j : Fin (n + 1 + 1)) (hij : i ≠ j) :
  dropPairEmb i j hij = dropPairEmb j i hij.symm

```

lemma permCond_dropPairEmb_symm[\[source\]](#)

```

@[simp]
lemma permCond_dropPairEmb_symm (i j : Fin (n + 1 + 1)) (hij : i ≠ j) :
  PermCond (c ∘ dropPairEmb i j hij) (c ∘ dropPairEmb j i hij.symm) id

```

lemma dropPairEmb_range[\[source\]](#)

```

@[simp]
lemma dropPairEmb_range {i j : Fin (n + 1 + 1)} (hij : i ≠ j) :
  Set.range (dropPairEmb i j hij) = {i, j}c

```

lemma dropPairEmb_image_compl[\[source\]](#)

```

lemma dropPairEmb_image_compl {i j : Fin (n + 1 + 1)} (hij : i ≠ j)
  (X : Set (Fin n)) :

```

$$(\text{dropPairEmb } i \ j \ \text{hij}) \ ' \ X^c = (\{i, j\} \cup \text{dropPairEmb } i \ j \ \text{hij} \ ' \ X)^c$$
lemma fst_neq_dropPairEmb_pre[\[source\]](#)

@[simp]

```
lemma fst_neq_dropPairEmb_pre (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin n) :
  ¬ i = (dropPairEmb i j hij) m
```

lemma snd_neq_dropPairEmb_pre[\[source\]](#)

@[simp]

```
lemma snd_neq_dropPairEmb_pre (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin n) :
  ¬ j = (dropPairEmb i j hij) m
```

def dropPairEmbPre[\[source\]](#)

Given an $i \ j \ m : \text{Fin } (n + 1 + 1)$ such that they are pairwise not equal, then dropPairEmbPre gives the pre-image of m under $\text{dropPairEmb } i \ j \ _$.

```
def dropPairEmbPre (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin (n + 1 + 1))
  (hm : m ≠ i ∧ m ≠ j) : Fin n
```

lemma dropPairEmb_dropPairEmbPre[\[source\]](#)

@[simp]

```
lemma dropPairEmb_dropPairEmbPre (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin (n + 1 + 1))
  (hm : m ≠ i ∧ m ≠ j) : dropPairEmb i j hij (dropPairEmbPre i j hij m hm) = m
```

lemma dropPairEmbPre_injective[\[source\]](#)

@[simp]

```
lemma dropPairEmbPre_injective (i j : Fin (n + 1 + 1)) (hij : i ≠ j)
  (m1 m2 : Fin (n + 1 + 1)) (hm1 : m1 ≠ i ∧ m1 ≠ j) (hm2 : m2 ≠ i ∧ m2 ≠ j) :
  dropPairEmbPre i j hij m1 hm1 = dropPairEmbPre i j hij m2 hm2 ↔ m1 = m2
```

lemma dropPairEmbPre_surjective[\[source\]](#)

```
lemma dropPairEmbPre_surjective (i j : Fin (n + 1 + 1)) (hij : i ≠ j)
  (m : Fin n) :
  ∃ m' : Fin (n + 1 + 1), ∃ (h : m' ≠ i ∧ m' ≠ j),
  dropPairEmbPre i j hij m' h = m
```

lemma dropPairEmb_comm[\[source\]](#)

```
lemma dropPairEmb_comm (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))
  (hij1 : i1 ≠ j1) (hij2 : i2 ≠ j2) :
  let i2'
```

lemma dropPairEmb_comm_apply[\[source\]](#)

```
lemma dropPairEmb_comm_apply (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))
  (hij1 : i1 ≠ j1) (hij2 : i2 ≠ j2) (m : Fin n) :
```

```
let i2'
```

lemma permCond_dropPairEmb_comm

[\[source\]](#)

```
@[simp]
lemma permCond_dropPairEmb_comm {n : ℕ} {c : Fin (n + 1 + 1 + 1 + 1) → S.C}
  (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))
  (hi1 : i1 ≠ j1) (hi2 : i2 ≠ j2) :
  let i2'
```

lemma eq_or_exists_dropPairEmb

[\[source\]](#)

```
lemma eq_or_exists_dropPairEmb
  (i j : Fin (n + 1 + 1)) (hi : i ≠ j) (m : Fin (n + 1 + 1)) :
  m = i ∨ m = j ∨ ∃ m', m = dropPairEmb i j hi m'
```

def dropPairOfMap

[\[source\]](#)

Given a bijection $\text{Fin } (n1 + 1 + 1) \rightarrow \text{Fin } (n + 1 + 1)$ and a pair $i j : \text{Fin } (n1 + 1 + 1)$, then $\text{dropPairOfMap } i j _ \sigma _ : \text{Fin } n1 \rightarrow \text{Fin } n$ corresponds to the induced bijection formed by dropping i and j in the source and their image in the target.

```
def dropPairOfMap {n n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hi : i ≠ j)
  (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : Function.Bijective σ)
  (m : Fin n1) : Fin n
```

lemma dropPairOfMap_injective

[\[source\]](#)

```
lemma dropPairOfMap_injective {n n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hi : i ≠ j)
  (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : Function.Bijective σ) :
  Function.Injective (dropPairOfMap i j hi σ hσ)
```

lemma dropPairOfMap_surjective

[\[source\]](#)

```
lemma dropPairOfMap_surjective {n n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hi : i ≠ j)
  (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : Function.Bijective σ) :
  Function.Surjective (dropPairOfMap i j hi σ hσ)
```

lemma dropPairOfMap_bijective

[\[source\]](#)

```
lemma dropPairOfMap_bijective {n n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hi : i ≠ j)
  (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : Function.Bijective σ) :
  Function.Bijective (dropPairOfMap i j hi σ hσ)
```

lemma permCond_dropPairOfMap

[\[source\]](#)

```
lemma permCond_dropPairOfMap {n n1 : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {c1 : Fin (n1 + 1 + 1) → S.C}
  (i j : Fin (n1 + 1 + 1)) (hi : i ≠ j)
  (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : PermCond c c1 σ) :
  PermCond (c ∘ dropPairEmb (σ i) (σ j)) (by simp [hσ.1.injective.eq_iff, hi]))
  (c1 ∘ dropPairEmb i j hi) (dropPairOfMap i j hi σ hσ.1)
```

def dropEm[\[source\]](#)

Given an ordered embedding $f : \text{Fin } m \hookrightarrow \text{Fin } n$ and a pure tensor p , $\text{dropEm } f \ p$ is the tensor formed by dropping all parts of p not in the image of f .

```
def dropEm {n : ℕ} {c : Fin n → S.C} {m : ℕ} (f : Fin m → Fin n) (p : Pure S c) : Pure S (c ∘ f)
```

def dropPair[\[source\]](#)

Given $i \ j : \text{Fin } (n + 1 + 1)$, $c : \text{Fin } (n + 1 + 1) \rightarrow S.C$ and a pure tensor $p : \text{Pure } S \ c$, $\text{dropPair } i \ j \ _p$ is the tensor formed by dropping the i th and j th parts of p .

```
def dropPair (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (p : Pure S c) :  
  Pure S (c ∘ dropPairEmb i j hij)
```

lemma dropPair_equivariant[\[source\]](#)

```
@[simp]  
lemma dropPair_equivariant {n : ℕ} {c : Fin (n + 1 + 1) → S.C}  
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (p : Pure S c) (g : S.G) :  
  dropPair i j hij (g • p) = g • dropPair i j hij p
```

lemma dropPair_symm[\[source\]](#)

```
lemma dropPair_symm (i j : Fin (n + 1 + 1)) (hij : i ≠ j)  
  (p : Pure S c) : dropPair i j hij p =  
  permP id (by simp) (dropPair j i hij.symm p)
```

lemma dropPair_comm[\[source\]](#)

```
lemma dropPair_comm {n : ℕ} {c : Fin (n + 1 + 1 + 1 + 1) → S.C}  
  (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))  
  (hij1 : i1 ≠ j1) (hij2 : i2 ≠ j2) (p : Pure S c) :  
  let i2'
```

lemma dropPair_update_fst[\[source\]](#)

```
@[simp]  
lemma dropPair_update_fst {n : ℕ} [inst : DecidableEq (Fin (n + 1 + 1))] {c : Fin (n + 1 + 1) →  
  S.C}  
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (p : Pure S c)  
  (x : S.FD.obj (Discrete.mk (c i))) :  
  dropPair i j hij (p.update i x) = dropPair i j hij p
```

lemma dropPair_update_snd[\[source\]](#)

```
@[simp]  
lemma dropPair_update_snd {n : ℕ} [inst : DecidableEq (Fin (n + 1 + 1))] {c : Fin (n + 1 + 1) →  
  S.C}  
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (p : Pure S c)  
  (x : S.FD.obj (Discrete.mk (c j))) :  
  dropPair i j hij (p.update j x) = dropPair i j hij p
```

lemma dropPair_update_dropPairEmb[\[source\]](#)

```
@[simp]
lemma dropPair_update_dropPairEmb {n : ℕ} [inst : DecidableEq (Fin (n + 1 + 1))]
  {c : Fin (n + 1 + 1) → S.C}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (p : Pure S c)
  (m : Fin n)
  (x : S.FD.obj (Discrete.mk (c (dropPairEmb i j hij m)))) :
  dropPair i j hij (p.update (dropPairEmb i j hij m) x) =
  (dropPair i j hij p).update m x
```

lemma dropPair_permP[\[source\]](#)

```
@[simp]
lemma dropPair_permP {n n1 : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {c1 : Fin (n1 + 1 + 1) → S.C} (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j)
  (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : PermCond c c1 σ) (p : Pure S c) :
  dropPair i j hij (permP σ hσ p) =
  permP (dropPairOfMap i j hij σ hσ.1) (permCond_dropPairOfMap i j hij σ hσ)
  (dropPair (σ i) (σ j) (by simp [hσ.1.injective.eq_iff, hij]) p)
```

def contrPCoeff[\[source\]](#)

Given a pure tensor $p : \text{Pure } S \ c$ and a $i \ j : \text{Fin } n$ corresponding to dual colors in c , $\text{contrPCoeff } i \ j \ _ \ p$ is the element of the underlying ring k formed by contracting p i and $p \ j$.

```
noncomputable def contrPCoeff {n : ℕ} {c : Fin n → S.C}
  (i j : Fin n) (hij : i ≠ j ∧ c i = S.τ (c j)) (p : Pure S c) : k
```

lemma contrPCoeff_permP[\[source\]](#)

```
@[simp]
lemma contrPCoeff_permP {n n1 : ℕ} {c : Fin n → S.C}
  {c1 : Fin n1 → S.C} (i j : Fin n1) (hij : i ≠ j ∧ c1 i = S.τ (c1 j))
  (σ : Fin n1 → Fin n) (hσ : PermCond c c1 σ) (p : Pure S c) :
  contrPCoeff i j hij (permP σ hσ p) =
  contrPCoeff (σ i) (σ j) (by simp [hσ.1.injective.eq_iff, hij, hσ.2]) p
```

lemma contrPCoeff_update_dropPairEmb[\[source\]](#)

```
@[simp]
lemma contrPCoeff_update_dropPairEmb {n : ℕ} [inst : DecidableEq (Fin (n + 1 + 1))]
  {c : Fin (n + 1 + 1) → S.C}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ c i = S.τ (c j)) (m : Fin n)
  (p : Pure S c) (x : S.FD.obj (Discrete.mk (c (dropPairEmb i j hij.1 m)))) :
  contrPCoeff i j hij (p.update (dropPairEmb i j hij.1 m) x) =
  contrPCoeff i j hij p
```

lemma contrPCoeff_update_fst_add[\[source\]](#)

```
@[simp]
lemma contrPCoeff_update_fst_add {n : ℕ} [inst : DecidableEq (Fin n)] {c : Fin n → S.C}
  (i j : Fin n) (hij : i ≠ j ∧ c i = S.τ (c j))
  (p : Pure S c) (x y : S.FD.obj (Discrete.mk (c i))) :
```

```

contrPCoeff i j hij (p.update i (x + y)) =
contrPCoeff i j hij (p.update i x) + contrPCoeff i j hij (p.update i y)

```

lemma contrPCoeff_update_snd_add[\[source\]](#)

```

@[simp]
lemma contrPCoeff_update_snd_add {n : ℕ} [inst : DecidableEq (Fin n)] {c : Fin n → S.C}
  (i j : Fin n) (hij : i ≠ j ∧ c i = S.τ (c j))
  (p : Pure S c) (x y : S.FD.obj (Discrete.mk (c j))) :
  contrPCoeff i j hij (p.update j (x + y)) =
  contrPCoeff i j hij (p.update j x) + contrPCoeff i j hij (p.update j y)

```

lemma contrPCoeff_update_fst_smul[\[source\]](#)

```

@[simp]
lemma contrPCoeff_update_fst_smul {n : ℕ} [inst : DecidableEq (Fin n)] {c : Fin n → S.C}
  (i j : Fin n) (hij : i ≠ j ∧ c i = S.τ (c j))
  (p : Pure S c) (r : k) (x : S.FD.obj (Discrete.mk (c i))) :
  contrPCoeff i j hij (p.update i (r • x)) =
  r * contrPCoeff i j hij (p.update i x)

```

lemma contrPCoeff_update_snd_smul[\[source\]](#)

```

@[simp]
lemma contrPCoeff_update_snd_smul {n : ℕ} [inst : DecidableEq (Fin n)] {c : Fin n → S.C}
  (i j : Fin n) (hij : i ≠ j ∧ c i = S.τ (c j))
  (p : Pure S c) (r : k) (x : S.FD.obj (Discrete.mk (c j))) :
  contrPCoeff i j hij (p.update j (r • x)) =
  r * contrPCoeff i j hij (p.update j x)

```

lemma contrPCoeff_dropPair[\[source\]](#)

```

lemma contrPCoeff_dropPair {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  (a b : Fin (n + 1 + 1)) (hab : a ≠ b)
  (i j : Fin n) (hij : i ≠ j ∧ c (dropPairEmb a b hab i) = S.τ (c (dropPairEmb a b hab j)))
  (p : Pure S c) : (p.dropPair a b hab).contrPCoeff i j hij =
  p.contrPCoeff (dropPairEmb a b hab i) (dropPairEmb a b hab j)
  (by simp using hij)

```

lemma contrPCoeff_symm[\[source\]](#)

```

lemma contrPCoeff_symm {n : ℕ} {c : Fin n → S.C} {i j : Fin n} {hij : i ≠ j ∧ c i = S.τ (c j)}
  {p : Pure S c} :
  p.contrPCoeff i j hij = p.contrPCoeff j i (hij.1.symm, by simp [hij])

```

lemma contrPCoeff_mul_dropPair[\[source\]](#)

```

lemma contrPCoeff_mul_dropPair {n : ℕ} {c : Fin (n + 1 + 1 + 1 + 1) → S.C}
  (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))
  (hij1 : i1 ≠ j1 ∧ c i1 = S.τ (c j1))
  (hij2 : i2 ≠ j2 ∧ c (dropPairEmb i1 j1 hij1.1 i2) = S.τ (c (dropPairEmb i1 j1 hij1.1 j2)))
  (p : Pure S c) :
  let i2'

```

lemma contrPCoeff_invariant[\[source\]](#)

```
@[simp]
lemma contrPCoeff_invariant {n : ℕ} {c : Fin n → S.C} {i j : Fin n}
  {hij : i ≠ j ∧ c i = S.τ (c j)} {p : Pure S c}
  (g : S.G) : (g • p).contrPCoeff i j hij = p.contrPCoeff i j hij
```

def contrP[\[source\]](#)

For $c : \text{Fin } (n + 1 + 1) \rightarrow S.C$, $i, j : \text{Fin } (n + 1 + 1)$ with dual color, and a pure tensor $p : \text{Pure } S \ c$, $\text{contrP } i \ j \ _p$ is the tensor (not pure due to the $n = 0$ case) formed by contracting the i th index of p with the j th index.

```
noncomputable def contrP {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ c i = S.τ (c j)) (p : Pure S c) :
  S.Tensor (c ∘ dropPairEmb i j hij.1)
```

lemma contrP_update_add[\[source\]](#)

```
@[simp]
lemma contrP_update_add {n : ℕ} [inst : DecidableEq (Fin (n + 1 + 1))] {c : Fin (n + 1 + 1) → S.C}
  (i j m : Fin (n + 1 + 1)) (hij : i ≠ j ∧ c i = S.τ (c j))
  (p : Pure S c) (x y : S.FD.obj (Discrete.mk (c m))) :
  contrP i j hij (p.update m (x + y)) =
  contrP i j hij (p.update m x) + contrP i j hij (p.update m y)
```

lemma contrP_update_smul[\[source\]](#)

```
@[simp]
lemma contrP_update_smul {n : ℕ} [inst : DecidableEq (Fin (n + 1 + 1))] {c : Fin (n + 1 + 1) → S.C}
  (i j m : Fin (n + 1 + 1)) (hij : i ≠ j ∧ c i = S.τ (c j))
  (p : Pure S c) (r : k) (x : S.FD.obj (Discrete.mk (c m))) :
  contrP i j hij (p.update m (r • x)) =
  r • contrP i j hij (p.update m x)
```

lemma contrP_equivariant[\[source\]](#)

```
@[simp]
lemma contrP_equivariant {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ c i = S.τ (c j)) (p : Pure S c) (g : S.G) :
  contrP i j hij (g • p) = g • contrP i j hij p
```

lemma contrP_symm[\[source\]](#)

```
lemma contrP_symm {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {i j : Fin (n + 1 + 1)} {hij : i ≠ j ∧ c i = S.τ (c j)} {p : Pure S c} :
  contrP i j hij p = permT id (by simp)
  (contrP j i (hij.1.symm, by simp [hij]) p)
```

def contrPMultilinear[\[source\]](#)

The multi-linear map formed by contracting a pair of indices of pure tensors.

```

noncomputable def contrPMultilinear {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ c i = S.τ (c j)) :
  MultilinearMap k (fun i => S.FD.obj (Discrete.mk (c i)))
    (S.Tensor (c ∘ dropPairEmb i j hij.1))where

```

def contrT[\[source\]](#)

For $c : \text{Fin } (n + 1 + 1) \rightarrow S.C$, $i j : \text{Fin } (n + 1 + 1)$ with dual color, and a tensor $t : \text{Tensor } S \ c$, $\text{contrT } i \ j \ _ \ t$ is the tensor formed by contracting the i th index of t with the j th index.

```

noncomputable def contrT {n : ℕ} {c : Fin (n + 1 + 1) → S.C} (i j : Fin (n + 1 + 1))
  (hij : i ≠ j ∧ c i = S.τ (c j)) :
  Tensor S c  $\rightarrow_l$ [k] Tensor S (c ∘ dropPairEmb i j hij.1)

```

lemma contrT_pure[\[source\]](#)

```

@[simp]
lemma contrT_pure {n : ℕ} {c : Fin (n + 1 + 1) → S.C} (i j : Fin (n + 1 + 1))
  (hij : i ≠ j ∧ c i = S.τ (c j)) (p : Pure S c) :
  contrT i j hij p.toTensor = p.contrP i j hij

```

lemma contrT_equivariant[\[source\]](#)

```

@[simp]
lemma contrT_equivariant {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ c i = S.τ (c j)) (g : S.G)
  (t : Tensor S c) :
  contrT i j hij (g • t) = g • contrT i j hij t

```

lemma contrT_permT[\[source\]](#)

```

lemma contrT_permT {n n1 : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {c1 : Fin (n1 + 1 + 1) → S.C}
  (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j ∧ c1 i = S.τ (c1 j))
  (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1))
  (hσ : PermCond c c1 σ) (t : Tensor S c) :
  contrT i j hij (permT σ hσ t) = permT (dropPairOfMap i j hij.1 σ hσ.1)
    (permCond_dropPairOfMap i j hij.1 σ hσ)
    (contrT (σ i) (σ j) (by simp [hσ.2, hij, hσ.1.injective.eq_iff]) t)

```

lemma contrT_symm[\[source\]](#)

```

lemma contrT_symm {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {i j : Fin (n + 1 + 1)} {hij : i ≠ j ∧ c i = S.τ (c j)} (t : Tensor S c) :
  contrT i j hij t = permT id (by simp)
    (contrT j i (hij.1.symm, by simp [hij]) t)

```

lemma contrT_comm[\[source\]](#)

```

lemma contrT_comm {n : ℕ} {c : Fin (n + 1 + 1 + 1 + 1) → S.C}
  (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))
  (hij1 : i1 ≠ j1 ∧ c i1 = S.τ (c j1))

```

```

(hij2 : i2 ≠ j2 ∧ c (dropPairEmb i1 j1 hij1.1 i2) = S.τ (c (dropPairEmb i1 j1 hij1.1 j2)))
(t : Tensor S c) :
let i2'

```

lemma dropPairEmb_succAbove[\[source\]](#)

```

lemma Pure.dropPairEmb_succAbove {n : ℕ}
  (i j : Fin (n + 1 + 1)) (hij : Fin (n + 1)) :
  dropPairEmb i (i.succAbove j) (Fin.ne_succAbove i j) =
  i.succAbove ∘ j.succAbove

```

lemma dropPairEmb_apply_lt_lt[\[source\]](#)

```

lemma Pure.dropPairEmb_apply_lt_lt {n : ℕ}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j)
  (m : Fin n) (hi : m.val < i.val) (hj : m.val < j.val) :
  dropPairEmb i j hij m = m.castSucc.castSucc

```

lemma dropPairEmb_natAdd_apply_castAdd[\[source\]](#)

```

lemma Pure.dropPairEmb_natAdd_apply_castAdd {n n1 : ℕ}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j)
  (m : Fin n1) :
  (dropPairEmb (n := n1 + n) (Fin.natAdd n1 i) (Fin.natAdd n1 j) (by simp_all [Fin.ne_iff_vne]))
  (Fin.castAdd n m)
  = Fin.castAdd (n + 1 + 1) (m)

```

lemma dropPairEmb_natAdd_image_range_castAdd[\[source\]](#)

```

lemma Pure.dropPairEmb_natAdd_image_range_castAdd {n n1 : ℕ}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j) :
  (dropPairEmb (n := n1 + n) (Fin.natAdd n1 i) (Fin.natAdd n1 j)
  (by simp_all [Fin.ne_iff_vne])) ''
  (Set.range (Fin.castAdd (m := n) (n := n1))) = {i | i.1 < n1}

```

lemma dropPairEmb_comm_natAdd[\[source\]](#)

```

lemma Pure.dropPairEmb_comm_natAdd {n n1 : ℕ}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j)
  (m : Fin n) :
  (dropPairEmb (n := n1 + n) (Fin.natAdd n1 i) (Fin.natAdd n1 j) (by simp_all [Fin.ne_iff_vne]))
  (Fin.natAdd n1 m)
  = Fin.natAdd (n1) (dropPairEmb i j hij m)

```

lemma dropPairEmb_permCond_prod[\[source\]](#)

```

lemma Pure.dropPairEmb_permCond_prod {n n1 : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {c1 : Fin n1 → S.C}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ c i = S.τ (c j)) :
  PermCond
  ((Sum.elim c1 c ∘ finSumFinEquiv.symm) ∘

```

```

(dropPairEmb (finSumFinEquiv (m := n1) (Sum.inr i))
 (finSumFinEquiv (m := n1) (Sum.inr j)) (by
  simp [hij, - finSumFinEquiv_apply_right, finSumFinEquiv.injective.eq_iff]))
(Sum.elim c1 (c ∘ ↑(dropPairEmb i j hij.1)) ∘ finSumFinEquiv.symm)
id

```

lemma contrPCoeff_natAdd[\[source\]](#)

```

lemma Pure.contrPCoeff_natAdd {n n1 : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {c1 : Fin n1 → S.C}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ c i = S.τ (c j))
  (p : Pure S c) (p1 : Pure S c1) :
  contrPCoeff (Fin.natAdd n1 i) (Fin.natAdd n1 j)
  (by simp_all [Fin.ext_iff]) (p1.prodP p)
  = contrPCoeff i j hij p

```

lemma contrPCoeff_castAdd[\[source\]](#)

```

lemma Pure.contrPCoeff_castAdd {n n1 : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {c1 : Fin n1 → S.C}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ c i = S.τ (c j))
  (p : Pure S c) (p1 : Pure S c1) :
  contrPCoeff (Fin.castAdd n1 i) (Fin.castAdd n1 j)
  (by simp_all [Fin.ext_iff]) (p.prodP p1)
  = contrPCoeff i j hij p

```

lemma prodP_dropPair[\[source\]](#)

```

lemma Pure.prodP_dropPair {n n1 : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {c1 : Fin n1 → S.C}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ c i = S.τ (c j))
  (p : Pure S c) (p1 : Pure S c1) :
  p1.prodP (dropPair i j hij.1 p) = permP id (Pure.dropPairEmb_permCond_prod i j hij)
  (dropPair (Fin.natAdd n1 i) (Fin.natAdd n1 j)
  (by simp_all [Fin.ext_iff]) (p1.prodP p))

```

lemma prodP_contrP_snd[\[source\]](#)

```

lemma Pure.prodP_contrP_snd {n n1 : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {c1 : Fin n1 → S.C}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ c i = S.τ (c j))
  (p : Pure S c) (p1 : Pure S c1) :
  prodT p1.toTensor (contrP i j hij p) =
  ((permT id (Pure.dropPairEmb_permCond_prod i j hij)) <|
  contrP
    (finSumFinEquiv (m := n1) (Sum.inr i))
    (finSumFinEquiv (m := n1) (Sum.inr j))
    (by simp [hij, - finSumFinEquiv_apply_right, finSumFinEquiv.injective.eq_iff]) <|
  prodP p1 p)

```

lemma prodT_contrT_snd[\[source\]](#)

```

lemma prodT_contrT_snd {n n1 : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {c1 : Fin n1 → S.C}

```

```

(i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ c i = S.τ (c j))
(t : Tensor S c) (t1 : Tensor S c1) :
prodT t1 (contrT i j hij t) =
((permT id (Pure.dropPairEmb_permCond_prod i j hij)) <|
contrT
  (finSumFinEquiv (m := n1) (Sum.inr i))
  (finSumFinEquiv (m := n1) (Sum.inr j))
  (by simp [hij, - finSumFinEquiv_apply_right, finSumFinEquiv.injective.eq_iff]) <|
prodT t1 t)

```

2.39 PhysLean.Relativity.Tensors.TensorSpecies.Tensor.Evaluation

[PhysLean/Relativity/Tensors/TensorSpecies/Tensor/Evaluation.lean](#) on GitHub.

def evalPCoeff

[\[source\]](#)

Given $a i : \text{Fin } (n + 1)$, $a b : \text{Fin } (S.\text{repDim } (c i))$ and a pure tensor $p : \text{Pure } S c$, $\text{evalPCoeff } i b p$ is the b th component of $p i$.

```
def evalPCoeff (i : Fin (n + 1)) (b : Fin (S.repDim (c i))) (p : Pure S c) : k
```

lemma evalPCoeff_update_self

[\[source\]](#)

```

@[simp]
lemma evalPCoeff_update_self (i : Fin (n + 1)) [inst : DecidableEq (Fin (n + 1))]
  (b : Fin (S.repDim (c i))) (p : Pure S c)
  (x : S.FD.obj (Discrete.mk (c i))) :
  evalPCoeff i b (p.update i x) = (S.basis (c i)).repr x b

```

lemma evalPCoeff_update_succAbove

[\[source\]](#)

```

@[simp]
lemma evalPCoeff_update_succAbove (i : Fin (n + 1)) [inst : DecidableEq (Fin (n + 1))]
  (j : Fin n)
  (b : Fin (S.repDim (c i))) (p : Pure S c)
  (x : S.FD.obj (Discrete.mk (c (i.succAbove j)))) :
  evalPCoeff i b (p.update (i.succAbove j) x) = evalPCoeff i b p

```

def evalP

[\[source\]](#)

Given $a i : \text{Fin } (n + 1)$, $a b : \text{Fin } (S.\text{repDim } (c i))$ and a pure tensor $p : \text{Pure } S c$, $\text{evalP } i b p$ is the tensor formed by evaluating the i th index of p at b .

```

noncomputable def evalP (i : Fin (n + 1)) (b : Fin (S.repDim (c i))) (p : Pure S c) :
  Tensor S (c ∘ i.succAbove)

```

lemma evalP_update_add

[\[source\]](#)

```

@[simp]
lemma evalP_update_add [inst : DecidableEq (Fin (n + 1))] (i j : Fin (n + 1))
  (b : Fin (S.repDim (c i))) (p : Pure S c)
  (x y : S.FD.obj (Discrete.mk (c j))) :
  evalP i b (p.update j (x + y)) =
  evalP i b (p.update j x) + evalP i b (p.update j y)

```

lemma evalP_update_smul[\[source\]](#)

```
@[simp]
lemma evalP_update_smul [inst : DecidableEq (Fin (n + 1))] (i j : Fin (n + 1))
  (b : Fin (S.repDim (c i))) (p : Pure S c)
  (r : k)
  (x : S.FD.obj (Discrete.mk (c j))) :
  evalP i b (p.update j (r • x)) =
  r • evalP i b (p.update j x)
```

def evalPMultilinear[\[source\]](#)

The multi-linear map formed by evaluation of an index of pure tensors.

```
noncomputable def evalPMultilinear {n : ℕ} {c : Fin (n + 1) → S.C}
  (i : Fin (n + 1)) (b : Fin (S.repDim (c i))) :
  MultilinearMap k (fun i => S.FD.obj (Discrete.mk (c i)))
  (S.Tensor (c ∘ i.succAbove)) where
```

def evalT[\[source\]](#)

Given $a i : \text{Fin } (n + 1)$, $a b : \text{Fin } (S.\text{repDim } (c i))$ and a tensor $t : \text{Tensor } S c$, $\text{evalT } i b t$ is the tensor formed by evaluating the i th index of t at b .

```
noncomputable def evalT {n : ℕ} {c : Fin (n + 1) → S.C} (i : Fin (n + 1))
  (b : Fin (S.repDim (c i))) :
  Tensor S c →[k] Tensor S (c ∘ i.succAbove)
```

2.40 PhysLean.Relativity.Tensors.Tree.Basic

[PhysLean/Relativity/Tensors/Tree/Basic.lean](#) on GitHub.

lemma field_eq_repr[\[source\]](#)

```
lemma field_eq_repr {c : Fin 0 → S.C} (t : TensorTree S c) :
  t.field = (S.tensorBasis c).repr t.tensor (fun j => Fin.elim0 j)
```

2.41 PhysLean.Relativity.Tensors.Tree.Elab

[PhysLean/Relativity/Tensors/Tree/Elab.lean](#) on GitHub.

def evalAdjustPos[\[source\]](#)

Adjusts a list $\text{List } \mathbb{N}$ by subtracting from each natural number the number of elements before it in the list which are less than itself. This is used to form a list of pairs which can be used for evaluating indices.

```
def evalAdjustPos (l : List ℕ) : List ℕ
```

def getEvalPos[\[source\]](#)

For list of indexExpr e.g. $[\alpha, 3, \beta, 2, \gamma]$, getEvalPos returns a list of pairs $\mathbb{N} \times \mathbb{N}$ related to indices which are numbers. The second element of each pair is the number corresponding to that index. The first element is the position of that number in the

list of indices when all other numbered indices before it are removed. Thus for the example given `getEvalPos` outputs $[(1, 3), (2, 2)]$.

```
def getEvalPos (ind : List (TSyntax `indexExpr)) : TermElabM (List (ℕ × ℕ))
```

def getContrPos

[\[source\]](#)

For list of `indexExpr` e.g. $[\alpha, 3, \beta, \alpha, 2, \gamma]$, `getContrPos` first removes all indices which are numbers (e.g. $[\alpha, \beta, \alpha, \gamma]$). It then outputs pairs (a, b) in $\mathbb{N} \times \mathbb{N}$ of positions of this list with $a < b$ such that the index at a is equal to the index at b . It checks whether or not an element is contracted more than once.

```
def getContrPos (ind : List (TSyntax `indexExpr)) : TermElabM (List (ℕ × ℕ))
```

def withoutContrEval

[\[source\]](#)

The list of indices after contraction or evaluation.

```
def withoutContrEval (ind : List (TSyntax `indexExpr)) : TermElabM (List (TSyntax `indexExpr))
```

def stringToTerm

[\[source\]](#)

The construction of an expression corresponding to the type of a given string once parsed.

```
def stringToTerm (str : String) : TermElabM Term
```

def getPermutationTerm

[\[source\]](#)

Given two lists of indices returns the permutation between them based on `finMapToEquiv`.

```
def getPermutationTerm (l1 l2 : List (TSyntax `indexExpr)) : TermElabM Term
```

def getNumIndicesExact

[\[source\]](#)

For syntax of the form T where T is `S.F.obj (OverColor.mk c)` this returns the value of `TensorSpecies.numIndices T`. That is, the exact number of indices associated with that tensor.

```
def getNumIndicesExact (stx : Syntax) : TermElabM ℕ
```

def getAllIndices

[\[source\]](#)

For syntax of the form $T \mid \alpha \beta \ 2 \ \beta$, `getAllIndices` returns a list $[\alpha, \beta, 2, \beta]$ of all `indexExpr`.

```
def getAllIndices (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

def getProdIndices

[\[source\]](#)

The function `getProdIndices` is defined for the following syntax: 1. For e.g. $T \mid \alpha \beta \ 2 \ \beta$, it returns all uncontracted and unevaluated indices e.g. $[\alpha]$ 2. For e.g. $T1 \mid \alpha \beta \ 2 \ \beta \otimes T2 \mid \alpha \gamma \ \delta \ \delta$ it returns all unevaluated indices which are not contracted

in either tensor e.g. $[\alpha, \alpha, \gamma]$. 3. For e.g. $(T1 \mid \alpha \beta \ 2 \ \beta \otimes T2 \mid \alpha \gamma \ \delta \ \delta) \otimes T3 \mid \gamma$ it does 2 recursively e.g. $[\gamma, \gamma]$

```
partial def getProdIndices (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

def getIndicesLeft

[\[source\]](#)

Gets the indices associated with the LHS of an addition.

```
def getIndicesLeft (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

def getIndicesRight

[\[source\]](#)

Gets the indices associated with the RHS of an addition.

```
def getIndicesRight (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

def getIndicesLeftEq

[\[source\]](#)

Gets the indices associated with the LHS of an equality.

```
def getIndicesLeftEq (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

def getIndicesRightEq

[\[source\]](#)

Gets the indices associated with the RHS of an equality.

```
def getIndicesRightEq (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

def nodeTermMap

[\[source\]](#)

For a term of the form T where T is $S.F.obj$ ($OverColor.mk \ c$), `tensorTermToTensorTree` returns the term corresponding to the `tensorNode T`

```
def nodeTermMap (T : Term) : Term
```

def evalTermMap

[\[source\]](#)

Given a list l of pairs $\mathbb{N} \times \mathbb{N}$ and a term T corresponding to a tensor tree, for each (a, b) in l , `evalSyntax` applies `TensorTree.eval a b` to T recursively. Here a is the position of the index to be evaluated and b is the value it is evaluated to.

For example, if l is $[(1, 2), (1, 4)]$ and T is a tensor tree then `evalSyntax l T` is `TensorTree.eval 1 4 (TensorTree.eval 1 2 T)`.

The list l is expected to be the output of `getEvalPos`.

```
def evalTermMap (l : List (N × N)) (T : Term) : Term
```

def contrTermMap

[\[source\]](#)

For each element of $l : List (\mathbb{N} \times \mathbb{N})$ applies `TensorTree.contr` to the given term.

```
def contrTermMap (l : List (N × N)) (T : Term) : Term
```

def prodTermMap [\[source\]](#)

The syntax associated with a product of tensors.

```
def prodTermMap (T1 T2 : Term) : Term
```

def negTermMap [\[source\]](#)

The syntax associated with a product of tensors.

```
def negTermMap (T1 : Term) : Term
```

def smulTermMap [\[source\]](#)

The syntax associated with the scalar multiplication of tensors.

```
def smulTermMap (c T : Term) : Term
```

def actionTermMap [\[source\]](#)

The syntax associated with the group action of tensors.

```
def actionTermMap (c T : Term) : Term
```

def addTermMap [\[source\]](#)

The syntax for a equality of tensor trees.

```
def addTermMap (permSyntax : Term) (T1 T2 : Term) : TermElabM Term
```

def equalTermMap [\[source\]](#)

The syntax for a equality of tensor trees.

```
def equalTermMap (permSyntax : Term) (T1 T2 : Term) : TermElabM Term
```

2.42 PhysLean.Relativity.Tensors.Tree.NodeIdentities.Basic

[PhysLean/Relativity/Tensors/Tree/NodeIdentities/Basic.lean](#) on GitHub.

lemma add_neg_neg [\[source\]](#)

```
lemma add_neg_neg (t1 t2 : TensorTree S c) :  
  (add (neg t1) (neg t2)).tensor = (neg (add t1 t2)).tensor
```

lemma smul_comm_neg [\[source\]](#)

```
lemma smul_comm_neg (a : k) (t : TensorTree S c) :  
  (smul a (neg t)).tensor = (neg (smul a t)).tensor
```

lemma action_zero[\[source\]](#)

```
lemma action_zero {c : Fin 0 → S.C} (g : S.G) (t : TensorTree S c) :
  (action g t).tensor = t.tensor
```

lemma action_field[\[source\]](#)

```
@[simp]
lemma action_field {c : Fin 0 → S.C} (g : S.G) (t : TensorTree S c) :
  (action g t).field = t.field
```

2.43 scripts.MetaPrograms.notes

[scripts/MetaPrograms/notes.lean](#) on GitHub.

def higgsPotential[\[source\]](#)

```
def higgsPotential : Note where
```

def tensors[\[source\]](#)

```
def tensors : Note where
```

def notesToMake[\[source\]](#)

```
def notesToMake : List (Note × String)
```

def makeYML[\[source\]](#)

```
def makeYML (nt : Note × String) : IO UInt32
```

2.44 scripts.MetaPrograms.TODO_to_yaml

[scripts/MetaPrograms/TOD0_to_yaml.lean](#) on GitHub.

inductive PhysLeanCategory[\[source\]](#)

```
inductive PhysLeanCategory
| ClassicalMechanics
| CondensedMatter
| Cosmology
| Electromagnetism
| Mathematics
| Meta
| Optics
| Particles
| QFT
| QuantumMechanics
| Relativity
| StatisticalMechanics
| Thermodynamics
| Other
```

def string[\[source\]](#)

```
def PhysLeanCategory.string : PhysLeanCategory → String
| ClassicalMechanics => "Classical Mechanics"
| CondensedMatter => "Condensed Matter"
| Cosmology => "Cosmology"
| Elctromagnetism => "Electromagnetism"
| Mathematics => "Mathematics"
| Meta => "Meta"
| Optics => "Optics"
| Particles => "Particles"
| QFT => "QFT"
| QuantumMechanics => "Quantum Mechanics"
| Relativity => "Relativity"
| StatisticalMechanics => "Statistical Mechanics"
| Thermodynamics => "Thermodynamics"
| Other => "Other"
```

def emoji[\[source\]](#)

```
def PhysLeanCategory.emoji : PhysLeanCategory → String
| ClassicalMechanics => "🔧"
| CondensedMatter => ""
| Cosmology => ""
| Elctromagnetism => "⚡"
| Mathematics => ""
| Meta => ""
| Optics => ""
| Particles => ""
| QFT => ""
| QuantumMechanics => "⚗️"
| Relativity => ""
| StatisticalMechanics => ""
| Thermodynamics => ""
| Other => ""
```

def List[\[source\]](#)

```
def PhysLeanCategory.List : List PhysLeanCategory
```

instance ToString PhysLeanCategory[\[source\]](#)

```
instance : ToString PhysLeanCategory where
```

def ofFileName[\[source\]](#)

```
def PhysLeanCategory.ofFileName (n : Name) : PhysLeanCategory
```

structure FullTODOInfo[\[source\]](#)

```
structure FullTODOInfo where
  content : String
  fileName : Name
  name : Name
  line : Nat
```

```
isInformalDef : Bool
isInformalLemma : Bool
category : PhysLeanCategory
```

def ofTODO[\[source\]](#)

```
def FullTODOInfo.ofTODO (t : todoInfo) : FullTODOInfo
```

def informalTODO[\[source\]](#)

```
def informalTODO (x : ConstantInfo) : CoreM FullTODOInfo
```

def allInformalTODO[\[source\]](#)

```
def allInformalTODO : CoreM (Array FullTODOInfo)
```

def toYAML[\[source\]](#)

```
def FullTODOInfo.toYAML (todo : FullTODOInfo) : MetaM String
```

def allTODOs[\[source\]](#)

```
unsafe def allTODOs : MetaM (List FullTODOInfo)
```

def categoriesToYML[\[source\]](#)

```
unsafe def categoriesToYML : MetaM String
```

def fullTODOYML[\[source\]](#)

```
unsafe def fullTODOYML : MetaM String
```